Master's Thesis

Master's Programme in Information Technology, 120 credits

Prototype Aligned Embedding for Time Series Forecasting

Computer Science and Engineering, 30 credits

Halmstad, December 30, 2025

Anila Shaji , Meghna Vijay

HALMSTAD UNIVERSITY

Anila Shaji
Meghna Vijay *:Prototype Aligned Embedding for Time Series Forecasting,*
© December 2025

ABSTRACT

Time-series forecasting plays a crucial role in supporting data-driven decisions across domains like energy systems, finance, and healthcare. Despite their strong predictive power, many deep learning models are still viewed as "black-boxes", making it difficult to understand how they arrive at their predictions. This lack of transparency often limits their broader adoption in real-world applications. To address this challenge, this thesis proposes an interpretable, prototype-based deep learning framework for multivariate time-series forecasting that integrates both numerical and textual modalities to enhance predictive accuracy and transparency.

At the core of the proposed framework lies the Prototype Similarity Network (ProSeNet), which learns a set of representative temporal patterns called prototypes that serve as interpretable anchors for model reasoning. Each forecast is explained in terms of similarity to these learned prototypes, providing clear insights into the model's decision process. To further enhance contextual understanding, the framework is extended into a multimodal architecture where textual embeddings derived from pretrained language models are fused with temporal features, enabling the model to reason jointly over structured and unstructured data.

Comprehensive experiments demonstrate that the proposed approach achieves strong predictive performance while maintaining interpretability. The prototype-based design allows users to trace predictions back to meaningful temporal patterns, and the integration of textual embeddings significantly improves contextual reasoning and accuracy. This work contributes a transparent, context-aware forecasting framework that bridges interpretable representation learning and multimodal reasoning advancing the development of trustworthy and explainable AI systems for time-dependent applications.

We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.

— Donald E. Knuth [18]

ACKNOWLEDGEMENTS

First, we express our sincere gratitude to our supervisors Yuantao Fan, Parisa Jamshidi, Nuwan Gunasekara and Adeel Zafar for their invaluable guidance, continuous support, and expertise throughout the course of this thesis. Their insightful feedback, constructive suggestions, and unwavering encouragement have been instrumental in shaping the quality, clarity, and direction of our research.

We also extend our thanks to Jens Lundstrom, our examiner, and Ali Amirahmadi, our opponent, for their constructive feedback and suggestions. Special thanks go to our professors at Halmstad University, whose teaching and mentorship have laid the foundation for our academic and professional growth. We are particularly grateful for the knowledge and skills imparted during our master's program, which have been pivotal in shaping this work.

We also acknowledge the support and patience of our family and friends, who have been a constant source of motivation throughout this journey. Their belief in our abilities has given us the strength to overcome challenges and stay focused on our goals.

Finally, we are grateful to the academic and professional communities whose research and innovations have inspired this work.

Thank you for your invaluable contributions and unwavering support.

CONTENTS

1 INTRODUCTION 1
- 1.1 Objective 2
- 1.2 Problem Statement 2
- 1.3 Research Questions 3
- 1.4 Novelty & Contribution 3
- 1.5 Evaluation 4
  - 1.5.1 Model Performance Evaluation 4
  - 1.5.2 Prototype Evaluation and Analysis 5
  - 1.5.3 Evaluation of Text Explanations 6

2 LITERATURE REVIEW 7
- 2.1 Background 7
- 2.2 Time Series Forecasting 8
- 2.3 Explainable AI (XAI) for Time Series Forecasting 9
- 2.4 Prototype based explanations 9
- 2.5 LLM-Based Explanations for XAI 10
- 2.6 Prompt Engineering 11

3 METHOD 13
- 3.1 Fundamental Concepts 13
  - 3.1.1 Time Series Forecasting 13
  - 3.1.2 Multivariate Time Series 13
  - 3.1.3 Forecasting Horizon 13
  - 3.1.4 Long Short-Term Memory (LSTM) 14
  - 3.1.5 Prototype Learning 14
  - 3.1.6 Embeddings 14
  - 3.1.7 Multimodal Learning 14
- 3.2 Model Architecture 15
  - 3.2.1 Encoder 15
  - 3.2.2 Prototype Layer 15
  - 3.2.3 LLM based Explanation Generation 17
  - 3.2.4 Multimodal Prototype-based Forecasting with Text Embeddings 18
- 3.3 Loss Functions 19
  - 3.3.1 Clustering Loss 19
  - 3.3.2 Evidence Loss 20
  - 3.3.3 Diversity Loss 20

4 DATASET DESCRIPTION 21
- 4.1 Real Dataset 21
- 4.2 Synthetic Dataset 21
- 4.3 Dataset Exploration through Prototype Learning 22
  - 4.3.1 What a Prototype Represents 23
  - 4.3.2 Feature-wise Prototype Analysis 24

5 EXPERIMENTAL EVALUATION 29

5.1 Experimental Setup 29
5.1.1 Data Preprocessing 29
5.1.2 Traning Setup 30
5.1.3 Hyperparameter Optimization 31
6 RESULTS 35
6.1 Classification on Synthetic Data 35
6.1.1 Model Evaluation Results 35
6.1.2 Learned Prototypes 37
6.1.3 Prototype Evaluation 38
6.1.4 LLM Generated Explanation of Predictions 41
6.1.5 Evaluation of Text Explanation 42
6.2 Classification on Real Data 42
6.2.1 Model Evaluation Results 42
6.2.2 Learned Prototypes 45
6.2.3 Prototype Evaluation 46
6.2.4 Sample Prediction and Prototype Similarity 49
6.2.5 LLM Generated Explanation of Predictions 49
6.2.6 Evaluation of Text Explanation 50
6.3 Forecasting Results 50
6.3.1 Model Performance Evaluation 50
6.3.2 Prototype Evaluation 52
6.3.3 Prototype Utilization Analysis 56
6.3.4 Sample Prediction and Prototype Similarity 57
6.3.5 LLM Generated Explanation of Predictions 59
6.3.6 Evaluation of Text Explanation 60
6.4 Multi-Modal Model Evaluation Results 61
6.5 Model Comparisons 63
7 DISCUSSION AND CONCLUSION 65
7.1 Discussion 65
7.2 conclusion 66
7.3 Limitations and future work 66

BIBLIOGRAPHY 69

LIST OF FIGURES

Figure 1 Architecture Diagram. 15
Figure 2 Multivariate time-series prototypes identified from the dataset using DTW-based `TimeSeriesKMeans`. 22
Figure 3 Feature importance for predicting prototype assignments in the ETT dataset. 24
Figure 4 Distribution of the HUFL feature across prototypes 25
Figure 5 Distribution of the HULL feature across prototypes 25
Figure 6 Distribution of the MUFL feature across prototypes 25
Figure 7 Distribution of the MULL feature across prototypes 26
Figure 8 Distribution of the LUFL feature across prototypes 26
Figure 9 Distribution of the LULL feature across prototypes 27
Figure 10 Training and validation cross-entropy loss and accuracy over epochs. 35
Figure 11 Auxiliary losses (clustering, evidence, and diversity) during training. 36
Figure 12 Confusion matrix of the classification results. The model achieved perfect performance, correctly classifying all instances of both classes without misclassification. 37
Figure 13 Sample predictions for 30 test inputs. The model correctly predicts all samples, demonstrating high accuracy as supported by the confusion matrix and other evaluation metrics. 37
Figure 14 Learned Prototypes from Synthetic data 39
Figure 15 Pairwise DTW distances between the learned prototypes. Lower values indicate stronger similarity, while higher values reflect distinct temporal behaviors. 39

Figure 16 Correlation matrix of learned prototypes, highlighting linear relationships. Negative correlations indicate opposing dynamics (e.g., Prototype 1 vs. 0/7 and 3/6), while low correlations (e.g., Prototypes 2 and 8) reveal unique temporal patterns distinct from other prototypes. 40
Figure 17 Training and validation cross-entropy loss and accuracy over epochs on ETT dataset 42
Figure 18 Auxiliary losses during training on ETT dataset 43
Figure 19 Confusion Matrix- ETT 44
Figure 20 Sample predictions for 30 test inputs for ETT dataset 45
Figure 21 Representation of learned Prototypes 46
Figure 22 DTW Matrix of performing classification on ETT dataset 48
Figure 23 Correlation Matrix of prototypes on ETT dataset 48
Figure 24 Dendrogram of performing classification on ETT dataset 49
Figure 25 Representation of input sample and nearest prototype 50
Figure 26 Training and validation loss curves for the ProSeNet forecasting model. 51
Figure 27 Auxiliary losses of forecasting(clustering, evidence, diversity) during training. 52
Figure 28 Visualization of the learned prototypes. 53
Figure 29 DTW matrix of learned prototypes. 53
Figure 30 Correlation matrix between forecasting prototypes, highlighting linear relationships and contrasting behaviors. 54
Figure 31 t-SNE visualization of inter-cluster relationships between prototype medoids based on DTW distances. Each point represents a prototype cluster medoid (C1–C5), and the relative proximity reflects similarity in temporal dynamics. 56
Figure 32 Prototype utilization summary showing the number of samples assigned to each prototype. 57
Figure 33 Comparison between a sample time-series sequence and its most similar prototype. The similarity score between them is 0.9993. 58

Figure 34 Comparison of predicted oil temperature (OT) for a sample sequence with its true OT values and the true OT pattern of the nearest prototype. The prototype similarity score is 0.9993. 59

LIST OF TABLES

Table 1 Distribution of samples across the learned prototypes for the ETT dataset. 23
Table 2 Sliding-window segmentation of the time series for forecasting the next 24 timesteps. 29
Table 3 Hyperparameter search space used for model tuning. 31
Table 4 Top-performing hyperparameter configurations based on validation MSE. 32
Table 5 Forecasting performance of the proposed model across different horizons. 33
Table 6 Forecasting performance of the model with different input sequence lengths. 33
Table 7 Upper-triangular inter-cluster distances between cluster medoids. Entries below the diagonal are omitted due to symmetry. 41
Table 8 Classification report of the experiment on ETT data. 45
Table 9 Upper-triangular inter-cluster DTW distances between medoids of prototype clusters on the ETT dataset. Entries below the diagonal are omitted due to symmetry. 47
Table 10 Optimal hyperparameter configuration used for final evaluation. 51
Table 11 Forecasting performance metrics of ProSeNet model. 52
Table 12 Inter-cluster DTW distances between cluster medoids of learned prototypes 55
Table 13 Quantitative metrics for evaluating the quality of the generated text explanation. 61
Table 14 Test Metrics for All 10 Runs of ProSeNetRegressorWithText 62
Table 15 Summary of Test Metrics Across 10 Runs (Mean $\pm$ Std) 62
Table 16 Performance comparison of different models on the test set. 63

ACRONYMS

AI Artificial Intelligence

AR Auto-Regressive

ARIMA Auto-Regressive Integrated Moving Average

BiLSTM Bidirectional Long Short-Term Memory

CNN Convolutional Neural Network

DTW Dynamic Time Warping

ETT Electricity Transformer Temperature

ETS Exponential Smoothing

GRU Gated Recurrent Unit

HUFL High Useful Load

HULL High Useless Load

LIME Local Interpretable Model-agnostic Explanations

LLM Large Language Model

LSTM Long Short-Term Memory

LUFL Low Useful Load

LULL Low Useless Load

MAE Mean Absolute Error

MSE Mean Squared Error

MUFL Medium Useful Load

MULL Medium Useless Load

NLP Natural Language Processing

OT Oil Temperature

PCA Principal Component Analysis

RAG Retrieval-Augmented Generation

ReLU Rectified Linear Unit

RMSE Root Mean Squared Error

R^2 Coefficient of Determination

SHAP SHapley Additive exPlanations

TSF Time Series Forecasting

TSPC Time Series Post-Correction

XAI Explainable Artificial Intelligence

1 INTRODUCTION

Time series forecasting [23] is a critical task in various domains, including energy consumption prediction, financial market analysis, and healthcare monitoring. Recent advances in deep learning models [20] [21], such as LSTM [45] networks and Transformer - based architectures [44], have significantly improved forecasting accuracy. However, the adoption of these models in real-world applications remains limited due to their "black-box" nature [20], which hinders trust and interpretability.

Explainable Artificial Intelligence (XAI) [34] [36] aims to address this challenge by making model predictions transparent and interpretable. Existing XAI techniques for time series [42] forecasting primarily rely on feature attribution methods such as SHAP (SHapley Additive exPlanations) [24] and LIME(Local Interpretable Model-agnostic Explanations) [35]. However, these methods often resulted in inconsistent and complex model explanations [26]. Prototype-based explanations, which involve identifying representative examples from historical data, offer a promising alternative by helping users understand model decisions through relatable past patterns [32]. Additionally, generating textual explanations enhances the interpretability of forecasts by presenting insights in natural language rather than relying solely on visual or numerical outputs [28].

This research presents a novel approach to time-series forecasting that integrates prototype-based interpretability with natural language explanations in a multimodal framework. The proposed model builds upon the Prototype Similarity Network (ProSeNet) [30] to identify representative temporal prototypes from historical data and map them to future forecasting patterns, thereby enabling interpretable justifications for its predictions. To enhance contextual understanding, the framework incorporates Natural Language Processing (NLP) [29] techniques to automatically generate human-readable explanations and fuses textual embeddings [15] with numerical time-series features in a shared latent space. This multimodal integration allows the model to reason jointly over structured and unstructured information, improving both predictive accuracy and interpretability. Ultimately, this hybrid design seeks to bridge the gap between accuracy and transparency, fostering greater trust and broader adoption of AI-driven forecasting in time-sensitive decision-making applications.

1.1 OBJECTIVE

The primary objective of this thesis is to develop an interpretable deep learning framework for time series forecasting by incorporating prototype-based explanations and natural language generation techniques. While modern forecasting models such as LSTMs and Transformers have achieved high predictive performance, they often function as "black boxes", offering limited transparency into their decision-making processes. This lack of interpretability poses a barrier to trust, adoption, and meaningful analysis, especially in domains where understanding the reasoning behind predictions is critical.

In this thesis, the focus will be on designing a prototype-based approach that identifies representative patterns from historical data and aligns them with forecasted sequences, enabling users to interpret predictions through relatable and contextual examples. In addition to visual pattern explanations, the framework will leverage NLP techniques to generate textual descriptions of the forecast rationale, making the results accessible to a wider range of users, including those without technical expertise.

Furthermore, the thesis will investigate how prototype-driven interpretability can influence and potentially enhance forecasting performance, by embedding explanation mechanisms within the model architecture. This hybrid approach aims to bridge the gap between model accuracy and transparency, thereby supporting more informed decision-making in real-world time-sensitive applications.

1.2 PROBLEM STATEMENT

Despite the advancements in deep learning-based time series forecasting, the lack of interpretability remains a major barrier to real-world adoption. Traditional post-hoc explainability techniques, like SHAP and LIME, use feature attributions to justify predictions. However, these methods may fall short in fully representing the decision-making process of the original model. As a result, models with built-in interpretability are often preferred in many applications [36].

Prototype-based explainability presents a more intuitive alternative by identifying past instances that resemble the current forecast, offering human-understandable justifications rooted in historical patterns. ProSeNet, a prototype-based method, has shown promise in aligning predictions with representative time series segments, providing a stable and interpretable approach compared to feature-based techniques. However, prototype-based methods alone may still require domain expertise to interpret retrieved examples effectively.

To enhance prototype explanations, this research integrates LLMs to generate human-readable textual justifications. By converting prototype-based insights into natural language narratives, LLMs make forecasting decisions more accessible, reducing the cognitive burden on users. This hybrid approach aims to improve transparency and trust in AI-driven forecasting models.

1.3 RESEARCH QUESTIONS

1. How can prototype-based explanations be utilized to enhance the interpretability of time series forecasting models?

   Prototype-based methods offer a case-based reasoning approach by linking model predictions to similar patterns observed in historical data. This question explores how such methods can make model behavior more transparent, enabling users to understand and trust forecasting outcomes by seeing familiar, context-relevant examples rather than abstract feature attributions.

2. How can prompt engineering be applied to enable LLMs to transform prototype explanations into human-readable texts?

   While prototype-based explanations provide useful reference patterns, they may still require expert knowledge to interpret. This question investigates the role of LLMs in translating these patterns into natural language narratives, making the insights more accessible and understandable to non-expert users.

3. How might explanatory insights be leveraged to improve the model's predictive accuracy?

   Beyond interpretability, there is growing interest in whether explanation mechanisms can positively influence model learning and performance. This question examines whether integrating explanation generation can guide the model toward more robust or generalizable forecasting, and how this impact can be measured.

1.4 NOVELTY & CONTRIBUTION

The primary contribution of this thesis is the development of a multimodal, prototype-based framework for explainable time-series forecasting that integrates numerical time-series representations and natural language information within a shared, interpretable latent space. While existing prototype-based methods such as ProSeNet are primarily designed for classification and rely solely on numerical representations, this work extends prototype-based reasoning to the forecasting domain and aligns learned temporal prototypes with seman-

tic information derived from textual descriptions. This alignment enables the model to ground its forecasts in both historical temporal patterns and contextual semantic knowledge. In addition to this core contribution, the thesis demonstrates how prototype-based reasoning can be translated into human-readable explanations by integrating a lightweight instruction-tuned language model, thereby improving transparency and accessibility for non-expert users. The proposed approach is implemented and validated on real-world time series datasets, demonstrating its potential to support more transparent, understandable, and effective forecasting in practical applications. The results demonstrate that the proposed approach improves transparency and interpretability while maintaining strong forecasting performance.

1.5 EVALUATION

1.5.1 *Model Performance Evaluation*

The performance of the model was evaluated on both the classification and forecasting task on datasets using a set of quantitative metrics and visual analyses.

LOSS CURVES: During training, the cross – entropy loss (classification) and MSE loss (forecasting) were monitored during training to ensure convergence. Auxiliary losses associated with prototype learning and reconstruction were also tracked to verify that the model effectively learned meaningful and diverse prototypes.

CLASSIFICATION METRICS: For the classification task, the ability of model to distinguish between the two oil temperature categories was assessed using:

- Accuracy – proportion of correctly classified samples.
- Precision, Recall, and F1-score – to evaluate the trade-off between false positives and false negatives.
- Confusion Matrix – to visualize class-wise prediction performance.

These metrics were computed on the test set using the `classification_report` function from the `scikit-learn` library. Together, they provide a baseline perspective on how well the model can discriminate between the classes.

FORECASTING METRICS: For the forecasting task, the predictive capability of model over multiple time horizons was evaluated using standard regression-based performance measures:

- Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{1}$$

- Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{2}$$

- Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{3}$$

- Coefficient of Determination (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2} \tag{4}$$

Here, y_i denotes the ground truth oil temperature, and $\hat{y}_i$ represents the model's forecast value. Lower MSE, MAE and RMSE values indicate better forecasting accuracy, while a higher R^2 value reflects stronger correlation between predicted and actual trends.

VISUAL EVALUATION: In addition to quantitative metrics, the predicted versus actual time series plots were analyzed to visually assess the quality of the model forecast.

Overall, these evaluation techniques provide a complete understanding of the behavior of the model in both classification and forecasting tasks.

1.5.2 *Prototype Evaluation and Analysis*

The learned prototypes were examined both quantitatively and qualitatively to ensure that they captured representative and diverse temporal patterns. The following analysis was conducted:

1. Visualization: Prototypes were plotted as multivariate time series, to highlight temporal variations across features.

2. Similarity Analysis: Dynamic Time Warping (DTW) distances between prototypes were calculated to find redundant prototypes.

3. Correlation Analysis: Pairwise correlation coefficients between prototypes were computed to assess redundancy.

4. Clustering Distances: Intra- and inter-cluster distances were calculated using hierarchical clustering with DTW distances to evaluate prototype diversity.

After analysis, redundant prototypes were removed, and the model was updated with distinct prototypes for final evaluation on the test dataset.

1.5.3 *Evaluation of Text Explanations*

The text explanations were analyzed based on four evaluation dimensions:

1 Length: measured by word count.

2 Completeness: checked against a predefined set of expected features.

3 Fluency & Grammar: evaluated using LanguageTool in Python. The number of grammar/fluency errors was recorded.

4 Readability: computed using the Flesch Reading Ease score.

This combination of quantitative and qualitative evaluation ensured that the explanations were not only accurate but also readable, grammatically correct, and feature-complete.

2 LITERATURE REVIEW

As the demand for accurate and interpretable time series forecasting grows across domains such as energy [2] [8], transportation, and healthcare, research has increasingly focused on balancing predictive performance with model transparency. Traditional statistical models offer interpretability but fall short in accuracy in complex settings, while modern deep learning models excel in accuracy, but often operate as opaque black boxes. This literature review explores key developments in time series forecasting, explainable AI methods especially prototype based and large language model enhanced explanations and highlights the need for approaches that combine data driven insights with human understandable reasoning.

2.1 BACKGROUND

Time series forecasting [23] [22] is a fundamental task in data science, where the goal is to predict future values based on historical temporal patterns. This technique plays a vital role in various industries, such as energy management [25], finance, transportation, and healthcare. Traditional statistical models like ARIMA and exponential smoothing have been widely used for forecasting due to their interpretability and mathematical simplicity. However, these models often struggle with complex, non-linear, and high-dimensional time series data found in real-world scenarios.

In recent years, deep learning models [6], particularly LSTM [45] networks and Transformer-based architectures, have shown significant improvements in forecasting accuracy by modeling long-range dependencies. These models are capable of capturing long-term dependencies and intricate temporal patterns that classical models cannot. Despite their performance advantages, deep learning models are often considered "black boxes" [20] due to their lack of interpretability, making it difficult for users to understand and trust their predictions.

To address this challenge, XAI has emerged as a critical research area focused on making AI models more transparent and interpretable [1]. Most existing XAI methods in time series forecasting rely on post-hoc feature attribution techniques [33], such as SHAP and LIME. While these methods attempt to explain the influence of input features on the model's predictions, they often produce inconsistent explanations, especially when applied to sequential data.

Beyond structural model improvements, researchers have explored post-processing techniques to enhance the accuracy of traditional mod-

els. One such development is the universal Time Series Post-Correction (TSPC) approach [17], which is specifically designed to mitigate errors in forecasts from models like ARIMA during long-horizon scenarios. Their method uses DTW to search historical data for shapes similar to a generated forecast and uses the historical "future" of those matches to apply a correction. This highlights how historical shape-matching can effectively improve predictive performance without modifying the underlying model architecture.

Prototype-based explanation [7] is a promising alternative in the XAI domain. Instead of explaining predictions through abstract feature importance scores, prototype-based methods justify decisions by referencing similar patterns from historical data. This approach aligns well with human reasoning, making it easier for users to relate new predictions to familiar examples. Models like ProSeNet and ProtoP-Net [13] have pioneered this line of work, particularly in image and sequence data.

The utility of this approach is particularly evident in industrial monitoring. Research conducted at Alfa Laval [43] applied these concepts to identify rare 'cleaning modes' in industrial separators, illustrating the practical value of motif-based identification. By combining (PCA) with the Matrix Profile algorithm, they isolated specific temporal motifs representing these infrequent states. Their introduction of a "pairwise similarity drift" method allows for the automatic determination of cluster sizes for these motifs, effectively distinguishing rare operational behaviors from normal, frequent patterns. This underscores the importance of motif-based reasoning in making sense of high-dimensional industrial time series.

With recent progress in LLMs, their integration into XAI tasks has become increasingly promising to further enhance interpretability that offer the ability to translate complex model behaviors into understandable narratives. By leveraging LLMs, prototype-based insights can be transformed into human-readable explanations, providing an intuitive interface for end-users to understand not only what the model predicts but also why.

This thesis builds on these foundations by combining prototype-based reasoning with LLM-generated explanations in the context of time series forecasting. This hybrid approach aims to improve model transparency while maintaining high predictive performance, ultimately making AI-driven forecasts more reliable and accessible.

2.2 TIME SERIES FORECASTING

Time series forecasting is the process of predicting future values based on historical data, with applications in energy, transportation, and climate modeling. Classical forecasting models, such as AR, ARIMA, and ETS, have long been used for predictive analytics. Although these

models are interpretable, they often struggle with complex, high-dimensional time-series data, limiting their accuracy in real-world scenarios [20].

Recent advances in deep learning have led to the development of powerful forecasting models [21]. These models leverage neural networks and attention mechanisms to capture long-term dependencies and nonlinear patterns in time series data. However, a significant drawback of deep learning-based forecasting methods is their lack of transparency, making it difficult for users to interpret predictions. This limitation has motivated research on XAI techniques to improve the interpretability of forecasting models [38].

2.3 EXPLAINABLE AI (XAI) FOR TIME SERIES FORECASTING

Explainable AI [1] [34] [25] [38] aims to enhance model transparency by providing insight into how predictions are made. Traditional post-hoc methods, such as SHAP and LIME, explain predictions by analyzing input feature contributions. However, these methods often suffer from instability, producing different explanations for the same input. Furthermore, they do not provide a direct reference to past cases, making it difficult to validate the explanations [32].

An alternative approach to XAI in time series forecasting is prototype-based explainability, where predictions are explained by referencing similar historical patterns. This method provides a case-based reasoning approach, improving user trust and interpretability.

2.4 PROTOTYPE BASED EXPLANATIONS

Prototype-based methods [9] aim to provide explanations by identifying representative past cases (prototypes) [11] that resemble the current input. This approach is inherently interpretable, as it allows users to compare new predictions with known historical patterns.

Recent advances in prototype-based neural networks have extended their use beyond static image classification into domains more similar to time-series forecasting and anomaly detection. The seminal work by Deep Learning for Case-Based Reasoning through Prototypes [19] introduced a hybrid autoencoder and prototype architecture which enables models to provide human-interpretable reasoning for each prediction by comparing latent representations to learned prototypes. Building on that, Explaining Deep Classification of Time-Series Data with Learned Prototypes [10] shows that prototypes remain effective when applied to sequential data, such as ECG or audio waveforms, demonstrating their potential in time-series contexts that relate to our forecasting task. Despite their promise, the interpretability of prototype-based models has been called into question, studies such as Evaluation and Improvement of Interpretability for Self-Explainable

Part-Prototype Networks [14] argue that similarity in latent space does not always correspond to human-understandable concepts, and propose objective metrics (e.g., consistency, stability) to evaluate explanation quality.

ProSeNet [30], a recent prototype-based model, learns a set of representative time series sequences and uses them to justify model decisions [32]. By showing which past time series segments are most similar to the current prediction, ProSeNet enables intuitive and transparent forecasting. Nevertheless, prototype-based frameworks remain flexible, for example, A Prototype-Based Neural Network for Image Anomaly Detection and Localization [12] successfully applies prototypes to detect and localize anomalies, showing their adaptability beyond classification tasks.

Unlike post-hoc explainability methods, prototype-based approaches are more stable, as they rely on actual data examples rather than computed feature attributions. However, one limitation of prototype-based methods is that they still require domain expertise to interpret the retrieved examples, which can be a barrier for non-experts. Recent research by [40] explores Text Prototype Aligned Embedding as a method to enhance LLMs' ability to process and explain time series data, demonstrating the potential of prototype-based approaches in improving interpretability and decision-making in forecasting models [16] [28].

Taken together, these studies support the theoretical basis of our approach, by learning prototypical patterns in latent space, a forecasting model can achieve both accurate predictions and interpretable representations.

2.5 LLM-BASED EXPLANATIONS FOR XAI

Recent advancements in LLMs have shown promising applications in XAI [31]. LLMs provide a solution by transforming complex model explanations into human-readable narratives, making AI-driven decisions more accessible and understandable [48].

Research has explored different ways LLMs enhance XAI [5]. [3] propose using LLMs [27] to refine explanations generated by existing XAI techniques into more natural and contextual narratives, which can improve user trust and comprehension. Additionally, [4] discuss how LLMs themselves require explainability, as they are complex black-box models. They argue that integrating XAI techniques into LLMs [41] is crucial for ensuring transparency and responsible AI deployment, especially in high-stakes applications like finance and healthcare [4].

2.6 PROMPT ENGINEERING

Prompt engineering refers to the systematic design and refinement of prompts task instructions, exemplars, or contextual templates used to guide LLMs toward producing accurate and relevant outputs without modifying their internal parameters. Recent surveys describe prompt engineering as both an art and a science of aligning model behavior with human intent, encompassing a wide taxonomy of methods such as zero-shot and few-shot prompting, chain-of-thought reasoning, retrieval-augmented generation (RAG), and automated prompt optimization [39]. From a practical standpoint, prompt engineering is critical for improving LLM robustness, interpretability, and fairness. The surveyed literature emphasizes that prompt performance is highly sensitive to exemplar selection, wording, and structure [37]. In TSF contexts, recent studies show that LLMs can be adapted for forecasting through structured prompt templates that incorporate temporal context, data descriptions, and forecasting goals [46].

Prompt engineering provides the essential bridge between numerical prediction and language-based explainability, enabling LLMs to produce more transparent and context-aware forecasts when integrated with prototype-driven and XAI-based time-series models.

3 METHOD

3.1 FUNDAMENTAL CONCEPTS

Before describing how the models were designed and trained, it is helpful to first introduce a few basic ideas that are used throughout this chapter. The methods in this work rely on concepts from multivariate time-series forecasting, segmentation of sequential data, the use of multiple data modalities, and learning prototype representations. Clarifying these ideas at the beginning makes it easier to follow the later sections, since the motivation behind each step of the methodology becomes more transparent. It also provides a clearer picture of the assumptions behind the modelling choices and why certain architectural components were included.

3.1.1 *Time Series Forecasting*

Time series forecasting refers to the task of predicting future observations based on historical data. Unlike static regression problems, time-series data are temporally ordered, which means the model must capture dependencies across time. Forecasting models learn to recognize patterns such as trends, seasonal variations, and temporal correlations. In this thesis, a sequence of past electrical load measurements is used to predict multiple future time steps.

3.1.2 *Multivariate Time Series*

A multivariate time series is a sequence of observations collected over time where multiple variables are recorded at each time step. Unlike univariate time series (which have only one variable), multivariate data captures interactions between multiple features. These variables often influence each other, and modelling their joint behaviour typically leads to more accurate forecasts.

3.1.3 *Forecasting Horizon*

The forecasting horizon refers to how far into the future the model predicts. In this thesis, the task is multi-horizon forecasting, meaning that multiple future time steps are predicted simultaneously rather than only the next value. This is important for practical planning, where future behaviour is needed over a window rather than a single point.

3.1.4 *Long Short-Term Memory (LSTM)*

LSTM networks introduce a gated architecture that explicitly controls information flow, enabling the model to capture long- short term temporal dependencies. Bidirectional LSTMs extend this by processing the sequence in both forward and backward directions, which improves representational capacity. LSTM networks form the backbone of the unimodal and multimodal architectures used in this thesis.

3.1.5 *Prototype Learning*

Prototype learning is based on the intuition that complex data can be represented using a set of representative examples, called prototypes. Instead of modelling every possible variation in the data, the model learns a compact set of vectors that summarize typical behaviour. During inference, the similarity between an input representation and the learned prototypes provides additional structure that improves interpretability and performance. In this thesis, prototypes are initialized using k-means clustering over latent time-series representations and are jointly optimized during training.

3.1.6 *Embeddings*

An embedding is a dense vector representation of information. Textual information is encoded using embeddings, which are dense numerical vectors obtained from pretrained language models. Semantically similar sentences are located close to each other in the embedding space. These embeddings are used as additional inputs to the model, allowing domain knowledge to influence the prediction process without manually crafting features.

3.1.7 *Multimodal Learning*

Multimodal learning combines information from different data sources or modalities. In this work, numerical time-series data are combined with textual information extracted from domain knowledge. The rationale is that the two modalities provide complementary signals: the time series captures dynamic behaviour and trends, and the textual data provides semantic context that can guide the model towards more meaningful representations. Integrating both sources enables the model to exploit richer information than from numerical data alone.

3.2 MODEL ARCHITECTURE

In this research, a prototype-based deep learning framework was developed for multivariate time-series analysis, beginning with a classification stage to identify temporal patterns, followed by extensions to forecasting tasks. The architecture was designed to capture both short-term and long-term dependencies while retaining interpretability through prototype learning.

The architecture (Fig 1) consists of three main components: an encoder that maps input sequences to latent embeddings, a prototype layer that enables interpretable representations, and a LLM for textual explanation generation.

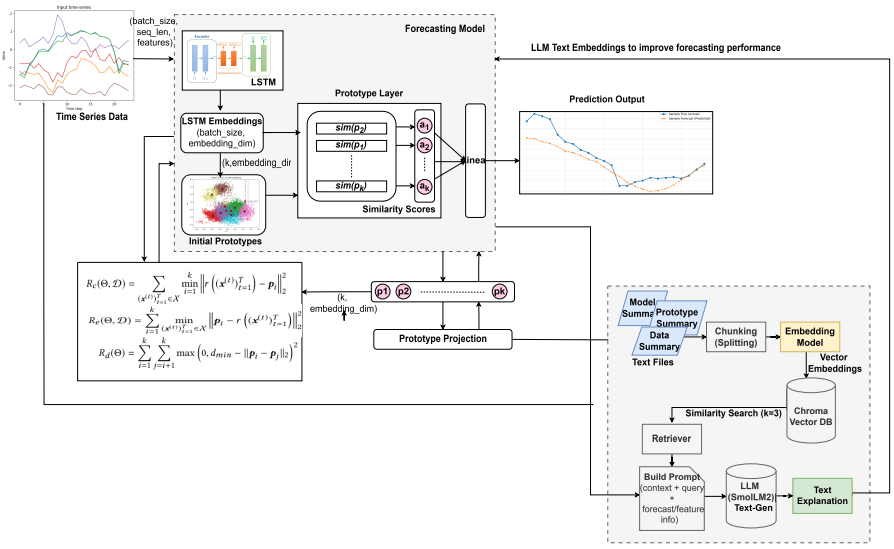

Figure 1: Architecture Diagram.

3.2.1 *Encoder*

The encoder's primary function is to transform raw input sequences into compact latent embeddings that capture essential temporal patterns. We employ a three-layer BiLSTM network with dropout regularization. The BiLSTM process the input in both forward and backward directions, producing hidden states that are combined into a fixed-size embedding vector for each sequence. These embeddings serve as a task-agnostic representation of the input and are used by the prototype layer to perform both classification and forecasting.

3.2.2 *Prototype Layer*

The prototype layer enhances interpretability by learning a set of representative latent embeddings, referred to as *prototypes*, which capture typical temporal patterns present in the data. These prototypes are

initialized using KMeans clustering applied to the latent embeddings produced by the encoder.

Given an input sequence, the encoder produces a latent embedding $\mathbf{h}$, which is compared against all learned prototypes $\{\mathbf{p}_i\}_{i=1}^{K}$. Pairwise Euclidean distances are computed using the ℓ_2 norm, and these distances are transformed into similarity scores using a Gaussian kernel:

$$d_i = \|\mathbf{h} - \mathbf{p}_i\|_2, \quad s_i = \exp(-d_i^2).$$

The resulting similarity vector $\mathbf{s} = [s_1, s_2, \ldots, s_K]$ explicitly quantifies how closely the input embedding matches each learned prototype.

1. Classification: In the classification setting, prototypes are class-specific and represent cluster centroids of latent embeddings associated with each class. The similarity vector $\mathbf{s}$ is directly passed to a fully connected layer, which applies a linear transformation to produce class logits:

   $$\mathbf{y} = \mathbf{W}\mathbf{s} + \mathbf{b}.$$

   This formulation allows the model to leverage prototype similarities and enabling predictions to be justified by identifying the most influential prototypes.

2. Forecasting: In the forecasting setting, prototypes are not class-specific but instead represent general temporal dynamics observed in the training data. Similarity scores between the input embedding and all prototypes are computed in the same manner as in classification. The similarity vector $\mathbf{s}$ is passed to a fully connected regression head:

   $$\hat{\mathbf{y}} = \mathbf{W}\mathbf{s} + \mathbf{b},$$

   where $\hat{\mathbf{y}}$ denotes the predicted future values over the forecasting horizon. This design grounds the forecast directly in a linear combination of learned prototype patterns, ensuring that predictions are rooted in representative historical dynamics.

In both classification and forecasting tasks, the fully connected layer acts as a linear aggregator over prototype similarity scores. This explicit structure allows each prototype's contribution to the final prediction to be directly interpreted through the learned weights, thereby strengthening the transparency and explainability of the proposed model.

3.2.3 *LLM based Explanation Generation*

To complement the quantitative predictions from ProSeNet, we employed a LLM to generate human-interpretable explanations of the classification and forecasting results. The LLM was prompted with the model's output along with information about the most similar prototype. Using this context, the LLM generated textual explanations that describe why the prediction was made, high lighting the relationship between input features and prototype behavior. The goal was to translate the similarity relations between input sequences and learned prototypes into concise, human-interpretable narratives.

The SmolLM2-1.7B-Instruct model was selected for explanation generation due to its strong balance between reasoning capability, instruction-following performance, and computational efficiency. Unlike large-scale proprietary LLMs, SmolLM2-1.7B can be deployed locally, ensuring reproducibility, data privacy, and suitability for academic experimentation. Its instruction-tuned design enables structured and coherent explanations while maintaining a relatively small parameter footprint, making it appropriate for integration into prototype-based forecasting systems without excessive computational overhead.

To enhance contextual grounding and reduce hallucinations, the LLM was combined with a Retrieval-Augmented Generation (RAG) pipeline implemented using LangChain. A summary covering the dataset, the model, and the prototype concept was stored in a Chroma vector database. This knowledge was embedded using MiniLM-L6 sentence embeddings, which were chosen for their efficiency and strong semantic retrieval performance. At inference time, a query prompt was constructed containing:

1. The model's forecast values or predicted class
2. The closest prototype index and its similarity score
3. The input sequence segment

Predictions along with related prototypes were retrieved from the knowledge base and embedded into the LLM prompt. The prompt was organized in a way that ensured explanations remained feature-specific and clearly linked to the corresponding prototypes.

The LLM then generated a short, structured explanation describing how input feature contributed to the similarity with the prototype and how this alignment informed the final prediction. This approach ensures interpretability by directly linking predictions to prototype comparisons while also producing explanations that are natural-language and user-friendly.

3.2.4 *Multimodal Prototype-based Forecasting with Text Embeddings*

To further enhance the forecasting performance of the prototype-based model, a multimodal extension was developed by integrating textual explanations directly into the learning process. This extended version combines temporal embeddings from time-series data with semantic embeddings derived from textual explanations.

3.2.4.1 *Encoder and Textual Embedding Fusion*

The encoder captures temporal dependencies within the input sequence $\mathbf{X}_t$ using a three-layer bidirectional LSTM. Each input sequence $\mathbf{X}_t \in \mathbb{R}^{T \times d}$ (where T is the time length and d is the number of features) is passed through the LSTM to obtain hidden states $\mathbf{h}_t$:

$$\mathbf{h} = [\overrightarrow{\mathbf{h}}_T; \overleftarrow{\mathbf{h}}_T] \in \mathbb{R}^{2H}$$

where H denotes the hidden dimension size. This bidirectional structure enables the encoder to capture both forward ($\overrightarrow{\mathbf{h}}_t$) and backward ($\overleftarrow{\mathbf{h}}_t$) temporal dependencies, allowing the model to learn comprehensive temporal representations from the input sequence.

To incorporate human-readable explanations, text data is first converted into dense vector representations using a pretrained Sentence-Transformer model (e.g., `all-MiniLM-L6-v2`) with an output dimension of 384. These text embeddings are then projected into the same latent space as the LSTM encoder through a linear transformation followed by a nonlinearity:

$$\mathbf{t}' = \mathrm{ReLU}(\mathbf{W}_t \mathbf{t} + \mathbf{b}_t)$$

where $\mathbf{t}' \in \mathbb{R}^{2H}$ represents the transformed text embedding, and $\mathbf{W}_t$, $\mathbf{b}_t$ are learnable parameters of the text projection layer.

The time-series representation $\mathbf{h}$ and the projected text embedding $\mathbf{t}'$ are concatenated and fused through a nonlinear transformation:

$$\mathbf{f} = \mathrm{ReLU}(\mathbf{W}_f[\mathbf{h}; \mathbf{t}'] + \mathbf{b}_f)$$

A Layer Normalization operation is then applied to $\mathbf{f}$ to ensure stable training and improved alignment between modalities. This fusion mechanism enables the model to jointly reason over both numerical and textual information, allowing contextual knowledge from the text domain to inform the temporal forecasting process.

3.2.4.2 *Prototype Layer and Forecasting Head*

The prototype mechanism in this model extends the principles of the base ProSeNet architecture. A set of K prototypes $\mathcal{P} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_K\}$

is initialized using K-Means clustering over the latent embeddings obtained from the encoder. These prototypes represent prototypical temporal–semantic patterns discovered in the training data. For each fused embedding $\mathbf{f}$, the similarity to each prototype is computed using a Gaussian kernel applied to the Euclidean distance between $\mathbf{f}$ and each prototype vector $\mathbf{p}_i$:

$$s_i = \exp\left(-\|f - \mathbf{p}_i\|^2\right)$$

The resulting similarity vector $\mathbf{s} = [s_1, s_2, \ldots, s_K] \in \mathbb{R}^K$ quantifies how closely the current input aligns with each learned prototype. These similarity scores are concatenated with the fused representation and passed to the forecasting head:

$$\mathbf{y} = \mathbf{W}_y[\mathbf{f}; \mathbf{s}] + \mathbf{b}_y$$

where $\mathbf{W}_y$ and $\mathbf{b}_y$ denote the weights and bias of the forecasting layer, respectively. This design allows the model to ground its predictions in both prototype similarity (interpretable reasoning) and semantic alignment (contextual understanding from text), thus enhancing forecasting accuracy.

3.3 LOSS FUNCTIONS

Both the classification and forecasting models share a common set of loss functions that regulate how prototypes interact with input embeddings. These losses were adapted from the formulation of [30] to ensure that the prototypes remain meaningful, diverse, and interpretable throughout the training.

3.3.1 *Clustering Loss*

The clustering loss ensures that input embeddings move closer to atleast one prototype. By minimizing the distance between each embedding and its nearest prototype, this loss encourages prototypes to capture representative patterns in the data. As training progresses, the embeddings naturally cluster around their most relevant prototypes, improving interpretability.

$$R_c(\Theta, \mathcal{D}) = \sum_{(\boldsymbol{x}^{(t)})_{t=1}^T \in \mathcal{X}} \min_{i=1}^{k} \left\| r\left((\boldsymbol{x}^{(t)})_{t=1}^T\right) - \boldsymbol{p}_i \right\|_2^2$$

3.3.2 *Evidence Loss*

The evidence loss guarantees that every prototype remains useful. Without this constraint, some prototypes could become unused. Through evidence loss, the model maintains prototype relevance by constraining every prototype to represent at least one input embedding, thereby preserving their contribution to prediction.

$$R_e(\Theta, \mathcal{D}) = \sum_{i=1}^{k} \min_{(\boldsymbol{x}^{(t)})_{t=1}^{T} \in \mathcal{X}} \left\| \boldsymbol{p}_i - r\left((\boldsymbol{x}^{(t)})_{t=1}^{T} \right) \right\|_2^2$$

3.3.3 *Diversity Loss*

Diversity loss prevents redundancy among prototypes. If multiple prototypes were too close to each other, they would represent overlapping patterns, reducing the diversity of explanations. This loss penalizes prototypes that are within a minimum distance threshold (d_min), encouraging them to spread out and capture distinct temporal behaviors.

$$R_d(\Theta) = \sum_{i=1}^{k} \sum_{j=i+1}^{k} \max\left(0, d_{min} - \|\boldsymbol{p}_i - \boldsymbol{p}_j\|_2\right)^2$$

In classification, these losses are combined with a standard cross–entropy loss to optimize both accuracy and interpretability.

In forecasting, they are combined with a regression loss to balance predictive accuracy with interpretability.

This approach bridges the gap between numerical outputs and interpretability by providing users with clear, descriptive narratives of the model's reasoning. Moreover, the LLM-based explanations were systematically evaluated using metrics-length, completeness, fluency, and readability to ensure both informativeness and linguistic quality.

4 DATASET DESCRIPTION

4.1 REAL DATASET

The Electricity Transformer Temperature (ETT) [47] dataset was used as the primary real-world dataset for this study. It consists of multivariate time series measurements recorded from electricity transformers over an extended operational period. The data set captures both load-related variables and the corresponding oil temperature measurements, providing a representative view of the behavior of the transformer under varying demand conditions.

Each record in the data set contains six primary input features: High Useful Load (HUFL), High Useless Load (HULL), Medium Useful Load (MUFL), Medium Useless Load (MULL), Low Useful Load (LUFL) and Low Useless Load (LULL). These features quantify electrical load levels in different operational categories, with 'useful' loads representing active, productive consumption, and 'useless' loads indicating nonproductive or auxiliary demand.

For the classification phase, each sequence was labeled into one of two categories:

High Oil Temperature -indicating elevated transformer oil temperature beyond the operational threshold.

Low Oil Temperature - representing low temperature conditions.

The labels were derived from the oil temperature measurements in the dataset, using a pre-defined threshold value to distinguish high-temperature events from normal operation. The classification task thus involved mapping historical multivariate load profiles to their corresponding temperature status.

The dataset exhibits a rich variety of temporal patterns, including step-like load increases (representing sudden demand surges) and oscillatory behaviors (reflecting fluctuating demand cycles). These patterns are particularly important for understanding the conditions that lead to thermal stress in transformers.

4.2 SYNTHETIC DATASET

For this work, we created a synthetic dataset that mimics the characteristics of the ETT dataset with same features and configurable number of samples. To simulate a similar environment while tailoring it for a time series classification task, a custom data generation function was implemented.

Each time series is generated using different waveform patterns– sine, cosine, square, inverse, or composite signals to provide diversity across the features. The signals are parameterized by random frequency, amplitude, phase, noise, and offset values, with variations depending on the class label. Specifically:

Class 0 (Low Oil Temperature): Signals have lower frequency, smaller amplitude, mild noise, and limited offset.

Class 1 (High Oil Temperature): Signals exhibit higher frequency, larger amplitude, stronger noise, and wider offset. Additionally, transient anomalies such as spikes, dips, or step changes are occasionally injected to mimic real-world transformer fluctuations under high stress.

This controlled design allows the dataset to preserve key temporal dynamics and non-linear behaviors seen in real transformer data, while ensuring clear separability between the classes for classification experiments. The final data set is stored as a tensor of shape (num_samples, sequence_length, num_features) along with a corresponding label vector.

4.3 DATASET EXPLORATION THROUGH PROTOTYPE LEARNING

To better understand the temporal structure within the ETT dataset, a prototype discovery experiment was conducted using the tslearn library. The goal was to identify representative time-series patterns (prototypes) that summarize typical operational modes of the transformer. The dataset contains 17,420 records and the multivariate data (HUFL, HULL, MUFL, MULL, LUFL, LULL, OT) was segmented into overlapping windows of 24 time steps using a sliding-window approach. Each window thus represents a daily-like sequence of transformer activity. Time-series clustering was applied (K=10), resulting in ten prototypes that characterize different load-temperature behaviors. Each cluster center (prototype) characterizes a distinct temporal regime across all seven features.

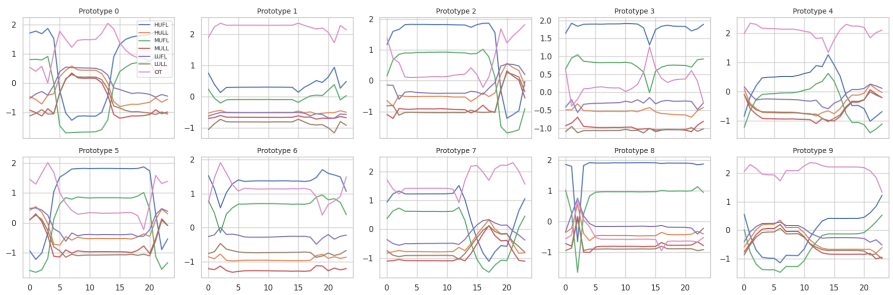

Figure 2: Multivariate time-series prototypes identified from the dataset using DTW-based `TimeSeriesKMeans`.

As illustrated in Figure 2, the prototypes effectively summarize the diverse daily operating behaviors of the system.

The cluster distribution was highly imbalanced, indicating dominant and rare operational regimes and shown in the below table 1:

Table 1: Distribution of samples across the learned prototypes for the ETT dataset.

Prototype ID	Number of Samples
0	2947
1	1409
2	1720
3	616
4	2024
5	758
6	2486
7	3399
8	285
9	1753

Visual inspection of the learned prototypes revealed diverse temporal motifs across the features. Some prototypes corresponded to stable load conditions, while others exhibited sharp fluctuations or transient spikes in load and temperature. Frequently occurring prototypes (e.g., IDs 0, 6, and 7) represent regular operating modes of the transformer under typical demand conditions. In contrast, rare prototypes (e.g., IDs 3 and 8) appear to capture abnormal or transitionary states characterized by abrupt changes in temperature or load.

4.3.1 *What a Prototype Represents*

In this context, a prototype is not a direct copy of one input feature (e.g., `HUFL` or `MULL`), but rather a *latent representation* summarizing the joint temporal behavior across all six load-related features and the oil temperature. Formally, each prototype P_i is represented as:

$$P_i \in \mathbb{R}^{T \times F}$$

where $T = 24$ is the sequence length (time steps per window), and $F = 6$ denotes the number of input features. Each prototype thus captures a typical multivariate pattern of transformer behavior over a daily cycle such as gradual load increases, stable low-load operation, or sudden high-load spikes leading to oil temperature rise.

This interpretation extends naturally to the forecasting model used later in this study, where prototypes function as interpretable latent references allowing the model to associate new input sequences with

known operational modes and to generate explainable forecasts grounded in these learned prototypes.

4.3.2 *Feature-wise Prototype Analysis*

To further understand which input features most strongly influence prototype formation, a feature importance analysis was conducted using a random forest classifier trained to predict prototype membership from the input sequences. The resulting importances, shown in Figure 3, indicate that all six load-related variables contribute significantly to defining temporal patterns, with `LUFL` and `LULL` being the most influential. This suggests that the lower load levels exhibit greater variability across operational modes and play a dominant role in differentiating prototypes.

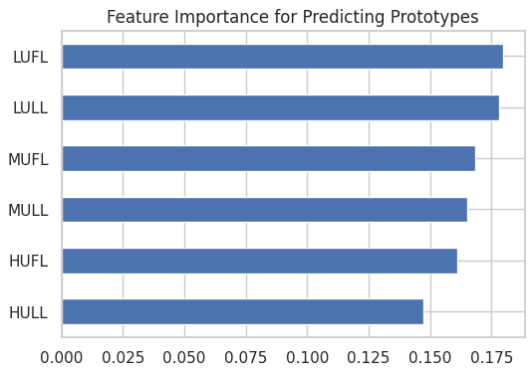

Figure 3: Feature importance for predicting prototype assignments in the ETT dataset.

To complement this, feature-wise boxplots were generated to visualize the statistical distribution of each variable across the learned prototypes. These plots provide a clearer understanding of how the mean, median, and variability of each feature differ between operational modes.

Figures 4–9 collectively demonstrate that each feature exhibits unique statistical signatures between prototypes.

Together, these analyses validate that the ETT dataset exhibits structured temporal behaviors, making it suitable for prototype-driven forecasting and explainable analysis, as explored in later chapters.

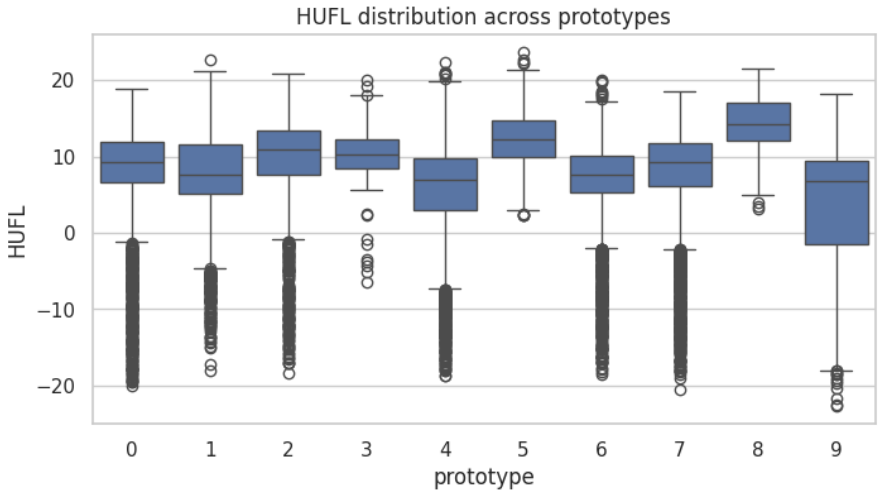

Figure 4: Distribution of the HUFL feature across prototypes

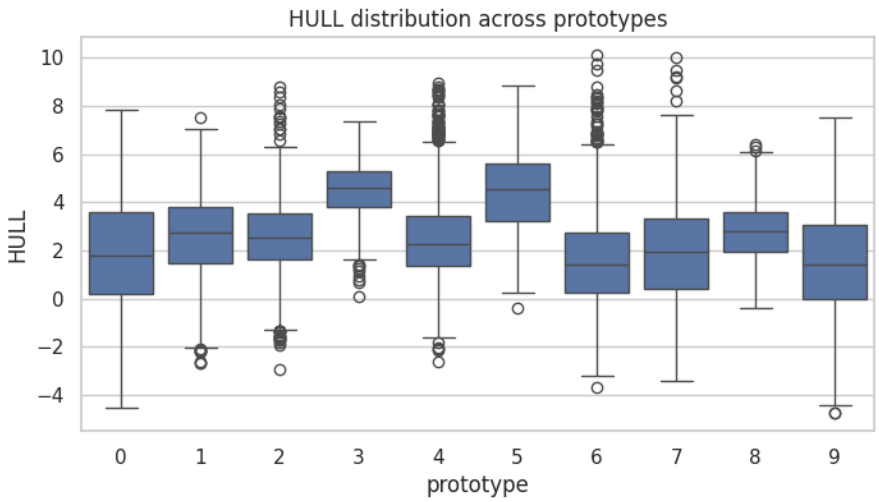

Figure 5: Distribution of the HULL feature across prototypes

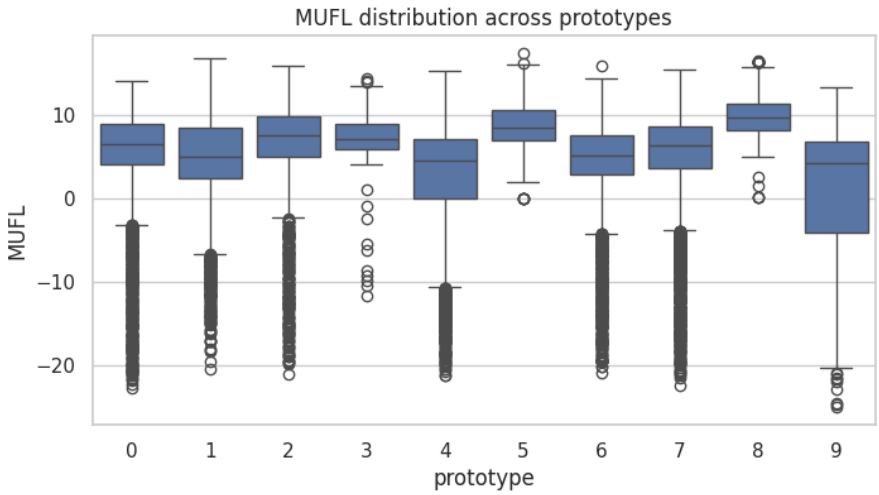

Figure 6: Distribution of the MUFL feature across prototypes

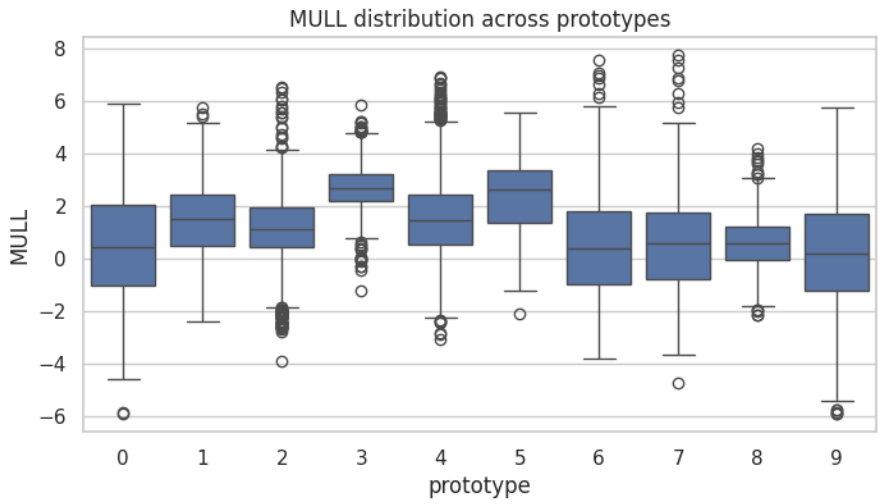

Figure 7: Distribution of the MULL feature across prototypes

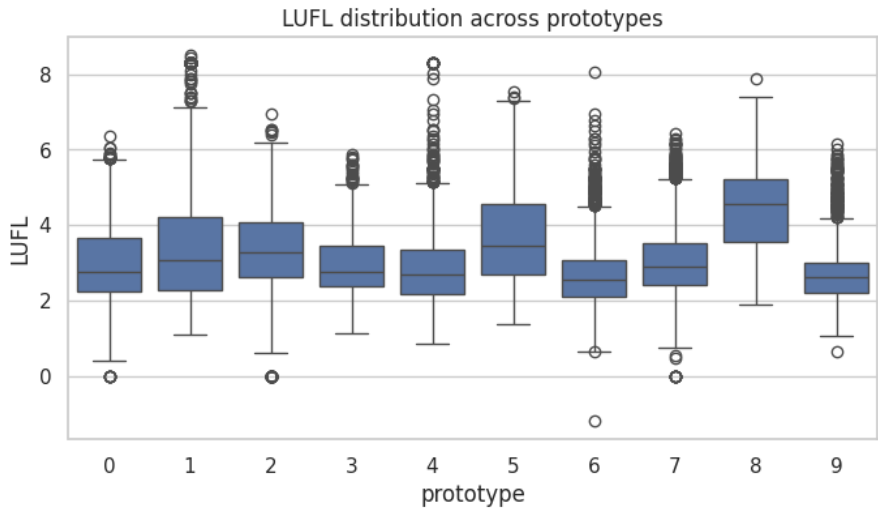

Figure 8: Distribution of the LUFL feature across prototypes

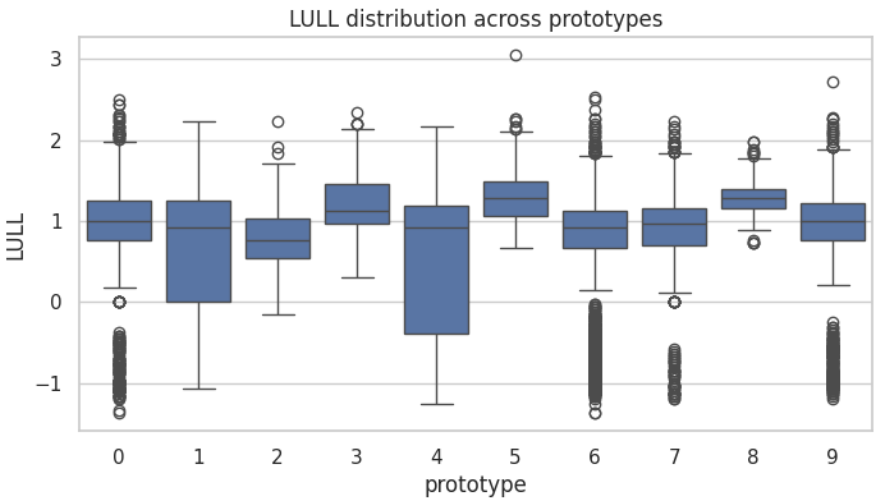

Figure 9: Distribution of the LULL feature across prototypes

5 EXPERIMENTAL EVALUATION

5.1 EXPERIMENTAL SETUP

In this section presents a comprehensive experimental evaluation of the proposed model. We conduct experiments on both synthetic and real-world sequence datasets to examine the model's performance in classification and forecasting tasks. The chapter is organized as follows. First, we describe the data preprocessing procedures and the training setup for classification experiments. Next, we outline the hyperparameter optimization strategies applied for forecasting tasks.

5.1.1 *Data Preprocessing*

Before initiating model training, the following preprocessing steps were applied:

5.1.1.1 *Segmentation*

Data segmentation is the process of dividing a long, continuous time series into smaller, fixed-length sequences (also called windows) that can be used as model inputs for training and forecasting. Deep learning models such as LSTMs and Transformers cannot directly process an entire continuous time series. Instead, they learn from shorter, overlapping windows that capture local temporal dependencies and seasonal patterns.

In this work, the ETT dataset was segmented into overlapping windows and each window of length $L = 24$ time steps was used as the model input, and the corresponding output consisted of the next $H = 24$ future time steps (the forecast horizon)(Table 2). This segmentation approach ensures that the model learns both short-term and periodic temporal patterns effectively.

Window	Input (past 24 timesteps)	Output (next 24 timesteps)
1	t_0–t_{23}	t_{24}–t_{47}
2	t_1–t_{24}	t_{25}–t_{48}

Table 2: Sliding-window segmentation of the time series for forecasting the next 24 timesteps.

5.1.1.2 *Normalization*

Normalization is a preprocessing technique that scales feature values into a common statistical range or distribution so that no single feature dominates the model training process. In the ETT dataset, the feature values vary significantly in magnitude. Without normalization, high-magnitude features could dominate gradient updates, resulting in biased learning. Z-score normalization (standardization) was applied using the `StandardScaler` from the `scikit-learn` library. For each feature x, normalization is defined as:

$$x_{norm} = \frac{x - \mu}{\sigma} \tag{5}$$

where μ and σ denote the mean and standard deviation of the corresponding feature, respectively.

Two independent scalers were used:

- Input scaler: Fitted on the training set and applied to input features.
- Target scaler: Fitted on the training target values and applied separately to forecast horizons.

5.1.1.3 *Data Splitting*

The dataset was divided into three subsets:

1. Training Set (64%): Used to train the model, this portion comprises the majority of the data and enables the model to learn representative temporal patterns.
2. Validation Set (16%): Used during model development to fine-tune hyperparameters and prevent overfitting by monitoring performance on unseen data.
3. Test Set (20%): Held out completely during training and validation, this set was used to evaluate the final model's ability to generalize to unseen sequences.

5.1.2 *Traning Setup*

5.1.2.1 *Classification*

The proposed model, ProSeNet, was implemented in PyTorch. The model was trained for 100 epochs with cross-entropy loss as the main criterion, in addition to auxiliary regularization terms. Training was performed using the Adam optimizer with the following learning rate and regularization hyperparameters were used:

Synthetic Data

1. lr = 0.001
2. λ_c = 0.01 (clustering regularization)
3. λ_e = 0.1 (evidence regularization)
4. λ_d = 0.01 (diversity regularization)

Real Data

1. lr = 0.001
2. λ_c = 0.5 (clustering regularization)
3. λ_e = 0.3 (evidence regularization)
4. λ_d = 0.1 (diversity regularization)

The model was configured to learn 10 prototypes, which serve as representative sequences to guide classification and provide explanations for the predictions.

5.1.3 *Hyperparameter Optimization*

5.1.3.1 *Forecasting Model Configuration*

The performance of deep learning models is highly sensitive to the choice of hyperparameters such as learning rate and regularization strength. To ensure fair comparison and optimal predictive capacity, we performed systematic hyperparameter tuning of the forecasting model. The goal was to identify the optimal configuration that minimizes the forecast error in the validation set while maintaining the stability and interpretability of the model.

The hyperparameter search space is summarized in the below table 3:

Hyperparameter	Symbol	Values Tested
Number of prototypes	k	{5, 8, 10, 15}
Learning rate	η	$\{1e^{-3}, 5e^{-4}, 1e^{-4}\}$
Clustering weight	λ_c	{0.1, 0.3, 0.5}
Evidence weight	λ_e	{0.1, 0.3, 0.5}
Diversity weight	λ_d	{0.05, 0.1, 0.3}

Table 3: Hyperparameter search space used for model tuning.

A grid search strategy was used to explore all possible combinations of the selected hyperparameters. For each combination, the model was trained for 50 epochs using the AdamW optimizer and MSE as the primary loss function. Training and validation performance were recorded for each epoch. After evaluating all runs, the configuration with the lowest validation MSE was selected for final testing.

Table 4 summarizes the best performing configurations based on the validation MSE.

Prototypes (k)	Learning Rate	λ_c	λ_e	λ_d	Val. MSE	Train MSE
5	0.001	0.1	0.1	0.05	**1.808**	1.929
15	0.001	0.3	0.3	0.05	1.886	2.053
5	0.001	0.1	0.1	0.3	1.887	1.951
15	0.001	0.1	0.1	0.05	1.891	1.896
8	0.001	0.1	0.1	0.05	1.903	2.011

Table 4: Top-performing hyperparameter configurations based on validation MSE.

Across all configurations tested, the model demonstrated stable convergence for learning rates between 0.0005 and 0.001. The best performing configuration, which achieved the lowest validation MSE of 1.808, was obtained with 5 prototypes, a learning rate of 0.001, a clustering weight λ_c of 0.1, an evidence weight λ_e of 0.1, and a diversity weight λ_d of 0.05. The corresponding training MSE for this configuration was 1.929.

This configuration was subsequently adopted for final model training and evaluation in the test set, which results are presented in the next chapter.

5.1.3.2 *Forecasting Performance Across Different Horizons*

To evaluate the temporal generalization capability of the forecasting model, We conducted experiments with multiple forecasting horizons: 12, 24, 48, and 72 timesteps. The model performance was assessed using four standard regression metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R^2). Table 5 summarizes the results.

The results in Table 5 show that the model achieves its best overall performance at the 12-step horizon, with the lowest MSE (1.5505), RMSE (1.2452), and MAE (0.8981), and the highest coefficient of determination ($R^2 = 0.9797$). This indicates that the model performs exceptionally well in short-term forecasting, where temporal dependencies from recent observations are strongest.

Table 5: Forecasting performance of the proposed model across different horizons.

Horizon	MSE	RMSE	MAE	R^2
12	1.5505	1.2452	0.8981	0.9797
24	1.6728	1.2934	0.9560	0.9769
48	1.7792	1.3339	0.9868	0.9758
72	1.7751	1.3323	1.0020	0.9770

As the forecasting horizon increases, there is an increase in error values can be observed (e.g., MSE rising from 1.55 to around 1.78), accompanied by a slight decrease in R^2. However, this decline is minimal, showing that the model maintains stable accuracy even for longer-term predictions. The performance consistency between the 24, 48, and 72-step horizons demonstrates that the model effectively captures medium to long range temporal dependencies without significant drift or instability.

Interestingly, at the longest horizon (72 steps), the model still retains strong predictive accuracy with $R^2 = 0.977$, nearly identical to shorter horizons. This confirms that the proposed model exhibits robust generalization over extended forecast windows, making it suitable for both short-term and longer-term forecasting tasks.

Overall, the 12-step horizon offers the highest precision for immediate forecasts, while the 24 to 72 step horizons maintain reliable accuracy with minimal performance decay, demonstrating the model's strong temporal generalization capability.

5.1.3.3 *Forecasting Performance Across Different Sequence Lengths*

To investigate the effect of historical context on forecasting accuracy, we trained and evaluated the model with varying input sequence lengths of 12, 24, 48, and 96 timesteps, while keeping the forecast horizon (24 timesteps) fixed. The performance metrics for each configuration are summarized in Table 6.

Table 6: Forecasting performance of the model with different input sequence lengths.

Sequence Length	MSE	RMSE	MAE	R^2
12	4.0173	2.0043	1.4324	0.9477
24	1.4794	1.2163	0.8860	0.9796
48	1.2097	1.0999	0.8079	0.9836
96	0.8553	0.9248	0.6791	0.9882

As observed from Table 6, the forecasting performance improves consistently with longer sequence lengths. When the model is provided with more historical data, it can better capture long-term temporal dependencies and seasonal trends, resulting in lower prediction errors across all metrics.

The coefficient of determination (R^2) increases steadily from 0.9477 for a 12-step input to 0.9882 for a 96-step input, while the Mean Absolute Error (MAE) decreases from 1.4324 to 0.6791. This trend indicates a substantial improvement in forecasting accuracy as the model gains access to a longer historical context. The most significant improvement occurs when extending the sequence length from 12 to 24 steps, suggesting that the model benefits strongly from incorporating at least one full day of past information.

Further extending the input length to 48 and 96 steps continues to yield incremental gains, demonstrating the model's ability to leverage extended temporal dependencies.

Overall, these results show that a sequence length of 96 timesteps provides the best forecasting performance, achieving the lowest error metrics and the highest R^2 value, while also demonstrating the LSTM model's strong capacity to utilize long-range temporal information.

6 RESULTS

In this section, we present the results of our proposed ProSeNet-based framework applied to the synthetic and Real (ETT) dataset on both classification and forecasting tasks.

6.1 CLASSIFICATION ON SYNTHETIC DATA

6.1.1 *Model Evaluation Results*

The learning dynamics of the model were tracked using cross-entropy loss and classification accuracy on both training and validation sets (Figure 10).

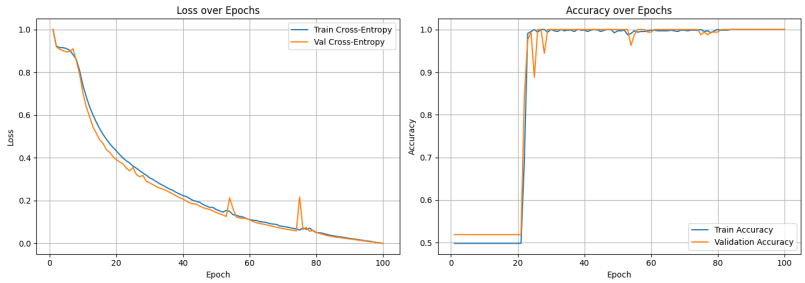

Figure 10: Training and validation cross-entropy loss and accuracy over epochs.

The training and validation cross-entropy losses decreased consistently over the course of training, with both curves showing a similar downward trajectory. This curves demonstrate that the model learned meaningful, class-separating representations without evidence of overfitting. Minor fluctuations in the validation loss around epochs 55-75 can be observed, but these were transient and quickly corrected as training progressed, eventually converging towards zero.

The accuracy curves further confirm the effectiveness of training. The model initially achieved random-level performance, but accuracy increased sharply after epoch 20, reaching nearly 100% accuracy for both training and validation sets. From epoch 25 onward, the model maintained stable performance with minimal gap between the training and validation accuracies, highlighting good generalization to unseen data.

Overall, the joint analysis of loss and accuracy demonstrates that the model converged effectively, avoided overfitting, and achieved perfect classification performance on the synthetic dataset.

Alongside cross-entropy loss, additional auxiliary losses were introduced to strengthen prototype learning and ensure interpretability. These include clustering, evidence, and diversity losses. The evolution of these losses during training is shown in Figure 11.

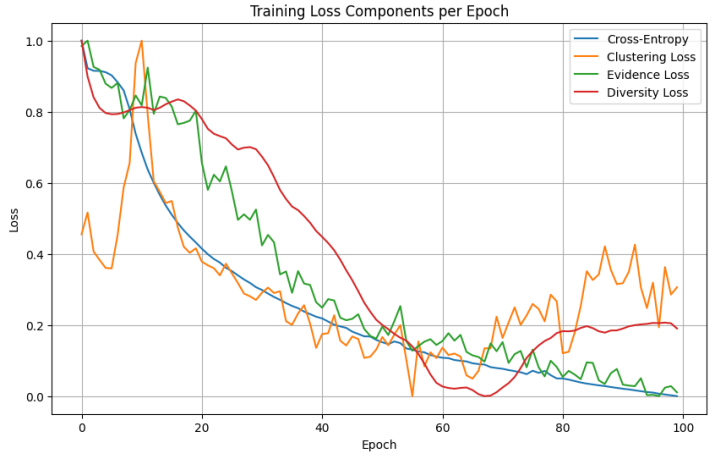

Figure 11: Auxiliary losses (clustering, evidence, and diversity) during training.

1. Clustering Loss
   The clustering loss showed fluctuations throughout training but followed a decreasing trend overall. This indicates that the data embeddings progressively moved closer to their nearest prototypes, ensuring that prototypes serve as meaningful representatives of the input space.

2. Evidence Loss:
   The evidence loss steadily decreased during training and approached near zero by the final epochs. This behavior suggests that all prototypes were actively utilized in classification, preventing redundancy where some prototypes might remain unused.

3. Diversity Loss:
   The diversity loss decreased during the initial epochs and then stabilized at a small positive value. This outcome confirms that prototypes retained sufficient separation from each other, avoiding collapse into redundant representations while still maintaining distinct temporal dynamics.

Taken together, the auxiliary losses validate that the prototypes learned by the model are representative (low clustering loss), fully engaged (low evidence loss), and diverse (stable diversity loss). These complementary objectives strengthen the interpretability of the model by ensuring that prototypes capture distinct and meaningful patterns in the synthetic dataset.

Figure 12 shows the confusion matrix for the classification results.The model achieved perfect classification performance, correctly identifying all samples from both classes without any misclassifications. Specifically, all instances of class 0 and class 1 were predicted accurately, yielding 99 true negatives and 101 true positives. This result highlights the effectiveness of the prototype-based learning framework and the auxiliary losses in establishing clear and discriminative decision boundaries.

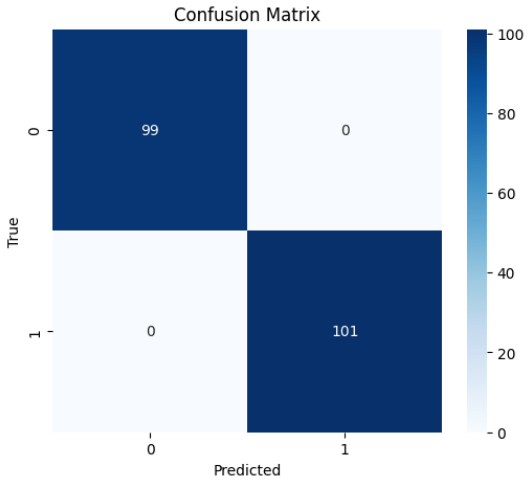

Figure 12: Confusion matrix of the classification results. The model achieved perfect performance, correctly classifying all instances of both classes without misclassification.

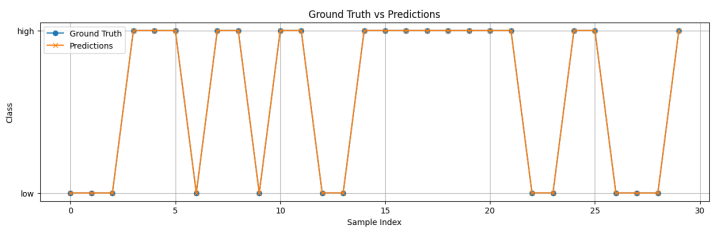

Figure 13: Sample predictions for 30 test inputs. The model correctly predicts all samples, demonstrating high accuracy as supported by the confusion matrix and other evaluation metrics.

To illustrate the performance of the model, predictions were generated for 30 sample inputs from the test dataset. As shown in Figure 13, all samples are correctly predicted, reflecting high accuracy of the model and its ability to generalize effectively to unseen data.

6.1.2 *Learned Prototypes*

Here is the visual representation of the ten prototypes learned by the model that represent characteristic temporal behaviors of the input

time series. These prototypes act as reference patterns that the model uses to compare against new sequences when making predictions.

As shown in Figure 14, the ten learned prototypes capture a diverse range of temporal dynamics across the input features (HUFL, MUFL, LUFL, HULL, MULL, LULL). Some of these prototypes exhibit similar trends, indicating recurring structures in the dataset, while others are more unique, capturing exceptional or contrasting dynamics.

Similar Prototypes:

1. Prototype 0 and Prototype 7 both show relatively smooth, low-frequency oscillations, corresponding to stable conditions under Low Oil Temperature (Class 0).
2. Prototype 3 and Prototype 6 emphasize consistently elevated, stable oscillations across multiple features, typical of sustained high-load or noisy states in Class 1.
3. Prototype 4 and Prototype 9 capture gradually rising oscillatory signals, indicating transitions toward increasing oil temperature conditions.

Unique Prototypes:

1. Prototype 2 stands out with persistent high-amplitude oscillations, aligning with high-frequency/high-temperature conditions (Class 1).
2. Prototype 5 is characterized by a sharp transient peak followed by a decline, representing spike or short-lived stress events in Class 1. Its behavior shows a similar pattern to Prototype 1.
3. Prototype 8 highlights mid-level oscillations with delayed increases, pointing to step-like changes or lagged responses in high-temperature conditions.

By learning these prototypes, the model captures both recurring stable patterns (seen in Class 0) and noisy, transient, or unstable behaviors (seen in Class 1). This makes predictions interpretable: when classifying a new sequence, the model identifies which prototype(s) the sequence most closely resembles, linking predictions directly to recognizable waveform dynamics.

6.1.3 *Prototype Evaluation*

From Fig 15, we observe that some prototypes are nearly identical (e.g., Prototype 0 and 7, Prototype 3 and 6, Prototype 4 and 9, Prototype 1 and 5 with distance $= 0.00$), while others such as Prototype 2 and Prototype 8 show consistently high distances with the rest, indicating that they capture unique temporal behaviors. This evaluation

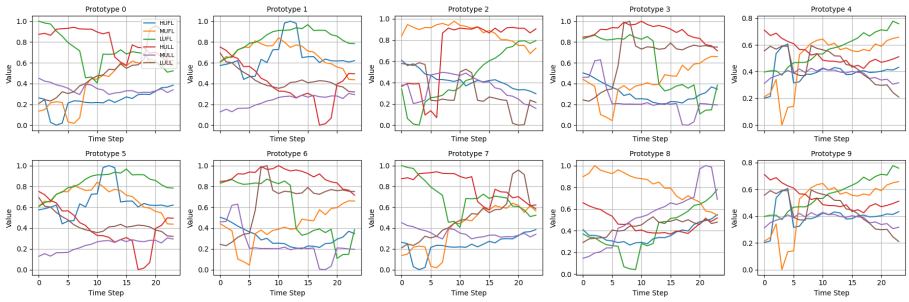

Figure 14: Learned Prototypes from Synthetic data

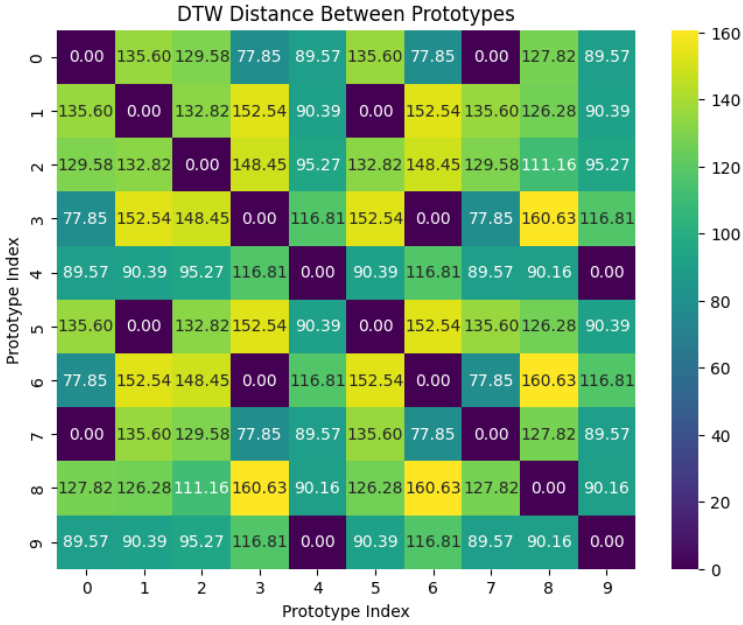

Figure 15: Pairwise DTW distances between the learned prototypes. Lower values indicate stronger similarity, while higher values reflect distinct temporal behaviors.

confirms that the prototype set includes both redundant patterns representing recurring stable dynamics and distinct patterns reflecting exceptional conditions.

To complement the DTW distance evaluation, a correlation matrix was computed to capture the linear relationships among prototypes as shown in Fig 16. Unlike DTW, which measures alignment in shape, correlation emphasizes whether prototypes move in the same or opposite direction.

Several prototype pairs show negative correlations, such as Prototype 1 with Prototypes 0/7 (-0.4) and Prototype 1 with Prototypes 3/6 (-0.42). These negative values indicate that when one prototype rises, the other tends to fall, revealing contrasting temporal behaviors.

Prototype 2 shows only weak correlations with the rest (between -0.29 and 0.29), reinforcing that it encodes a distinct temporal pattern that does not linearly align with other prototypes.

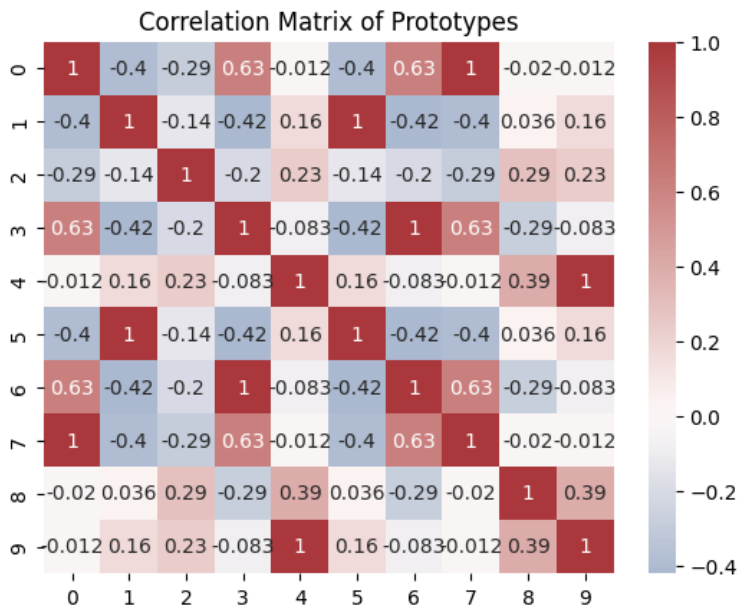

Figure 16: Correlation matrix of learned prototypes, highlighting linear relationships. Negative correlations indicate opposing dynamics (e.g., Prototype 1 vs. 0/7 and 3/6), while low correlations (e.g., Prototypes 2 and 8) reveal unique temporal patterns distinct from other prototypes.

Prototype 8 similarly exhibits low to modest correlations (up to 0.39 with 4/9), confirming it as another unique reference pattern in the learned set.

Thus, while the DTW analysis highlighted structural similarity and redundancy, the correlation analysis emphasizes prototypes with opposing or unaligned behaviors, particularly pointing to the uniqueness of Prototypes 2 and 8. This dual evaluation confirms that the prototype set covers both common waveform shapes and contrasting dynamics, making the model's explanations richer and more interpretable.

To evaluate the quality of the learned prototypes, hierarchical clustering was applied using DTW as a similarity measure. The clustering quality was assessed through both intra-cluster and inter-cluster distances.

Intra-cluster distances reflect the compactness of clusters, i.e., how similar the prototypes within the same cluster are. In this experiment, all clusters exhibited an intra-cluster distance of 0.00, which indicates that the prototypes within each cluster are identical under the DTW measure. This result is particularly meaningful for clusters with multiple members (e.g., Cluster 1: [0, 7], Cluster 2: [3, 6], Cluster 3: [1, 5], Cluster 4: [4, 9]), where the distance confirms that the grouped prototypes share highly consistent temporal patterns. For clusters contain-

ing only a single prototype (Cluster 5: [8], Cluster 6: [2]), the intra-cluster distance is trivially zero, as no pairwise comparison exists.

Inter-cluster distances measure the separation between different clusters. Table 7 presents the results with values ranging from 77.85 (between Cluster 1 and Cluster 2) to 160.63 (between Cluster 2 and Cluster 5). These high values indicate that the prototypes in different clusters are well separated from each other, ensuring that the clusters represent distinct temporal dynamics.

	C1	C2	C3	C4	C5	C6
C1	-	77.85	135.60	89.57	127.82	129.58
C2		-	152.54	116.81	160.63	148.45
C3			-	90.39	126.28	132.82
C4				-	90.16	95.27
C5					-	111.16
C6						-

Table 7: Upper-triangular inter-cluster distances between cluster medoids. Entries below the diagonal are omitted due to symmetry.

6.1.4 *LLM Generated Explanation of Predictions*

To improve the interpretability of the proposed ProSeNet model, we provide an explanation of a prediction for a given test instance.

Prediction: The model predicts *High Oil Temperature (Class 1)*.

Explanation: *The Synthetic Electricity Transformer model predicts High Oil Temperature (class 1) because the instance's features (HUFL, HULL, MUFL, MULL, LUFL, and LULL) are similar to the prototype for this class. This similarity is reflected in prototype similarity scores, which indicate how closely the instance's features match the learned patterns in the training data.*

The input features (HUFL, HULL, MUFL, MULL, LUFL, and LULL) in the instance are high–amplitude, high–frequency, and high–noise signals, much like the ones observed in the dataset for High Oil Temperature. These features align with the prototype's characteristics of high oil temperature, as they exhibit high energy and rapid frequency changes.

The instance's features are more similar to the prototype than the ones from class 0, which are low–amplitude, low–frequency, and low–noise signals. This suggests that the instance is more likely to belong to class 1, based on its input features. The prediction of High Oil Temperature is further supported by the similarity scores, which indicate that the instance's features are very close to those of the prototype for class 1. This alignment with the prototype explains why the model predicted High Oil Temperature for the instance.

This case study demonstrates that the model prediction is not only accurate but also interpretable, as the reasoning is supported by similarity to learned prototypes.

6.1.5 *Evaluation of Text Explanation*

The generated explanation has a length of 191 words, which is sufficient to provide detailed reasoning without being excessively long. It achieved the maximum completeness score of 2, indicating that the explanation fully covers the relevant features that contributed to the prediction. While the analysis identified 7 minor fluency or grammar errors, these do not significantly affect the overall clarity. There is a room to improve the readability score of the explanation as it gives the score of 42.6, suggests the text is moderately complex, suitable for a technical audience familiar with the domain. Overall, the explanation is comprehensive, well-structured, and informative, making it a strong support for model interpretability despite minor language issues.

6.2 CLASSIFICATION ON REAL DATA

6.2.1 *Model Evaluation Results*

The learning dynamics of the model on the ETT dataset were tracked using cross-entropy loss and classification accuracy on both training and validation sets (Figure 17).

During training, the cross-entropy losses for both the training and validation sets decreased consistently, following similar downward trend. This indicates that the model successfully learned discriminative temporal representations without clear signs of overfitting. Although the validation loss exhibited fluctuations during later epochs, these remained within a stable range, suggesting the robustness of the optimization process. The accuracy of both the training and validation sets has increased consistently throughout the training process.

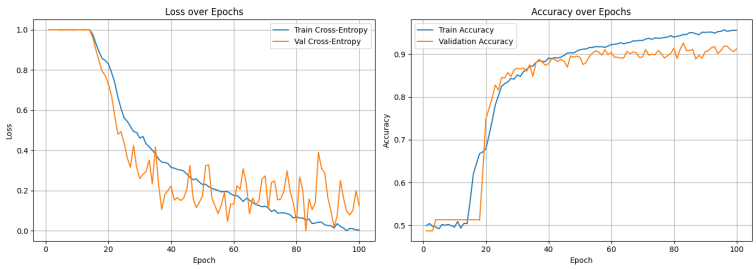

Figure 17: Training and validation cross-entropy loss and accuracy over epochs on ETT dataset

In addition to the cross-entropy loss, the training process also incorporated auxiliary objectives to improve the quality and interpretability of learned prototypes. The decomposition of the total loss into clustering, evidence, and diversity components is shown in Figure 18.

1. Clustering Loss
   The clustering loss decreased overall as training progressed, despite local oscillations. This suggests that input embeddings were gradually drawn closer to their assigned prototypes, reinforcing the role of prototypes as meaningful representatives of temporal patterns in the data.

2. Evidence Loss
   The evidence loss steadily declined towards small values, confirming that all prototypes remained active contributors during classification. This prevented prototype redundancy and ensured that each prototype encoded distinctive aspects of the time series.

3. Diversity Loss
   The diversity loss quickly dropped in the early epochs and stabilized thereafter at a low positive value. This indicates that prototypes were sufficiently separated from one another, avoiding collapse into redundant representations while still maintaining complementary temporal dynamics.

Taken together, the loss dynamics on the ETT dataset confirm that the prototypes learned by the model are compact (low clustering loss), engaged (low evidence loss), and diverse (stable diversity loss). These complementary objectives ensured that the model not only achieved strong predictive performance but also retained interpretable and meaningful prototype representations of the ETT data.

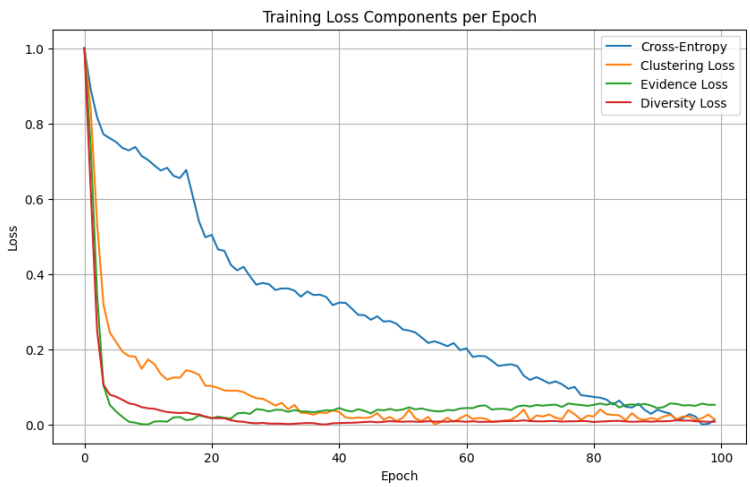

Figure 18: Auxiliary losses during training on ETT dataset

Figure 19 shows the confusion matrix for the classification results in the ETT data set. Of the 1,747 samples belonging to class 0, the model correctly classified 1,572, while 175 were misclassified as class 1. Similarly, for the 1,733 samples from class 1, a total of 1,646 were correctly predicted, with 87 misclassified as class 0. The strong concentration of values along the diagonal highlights the model's ability to effectively capture the temporal patterns of both classes, while the relatively small off-diagonal counts indicate only limited confusion between them.

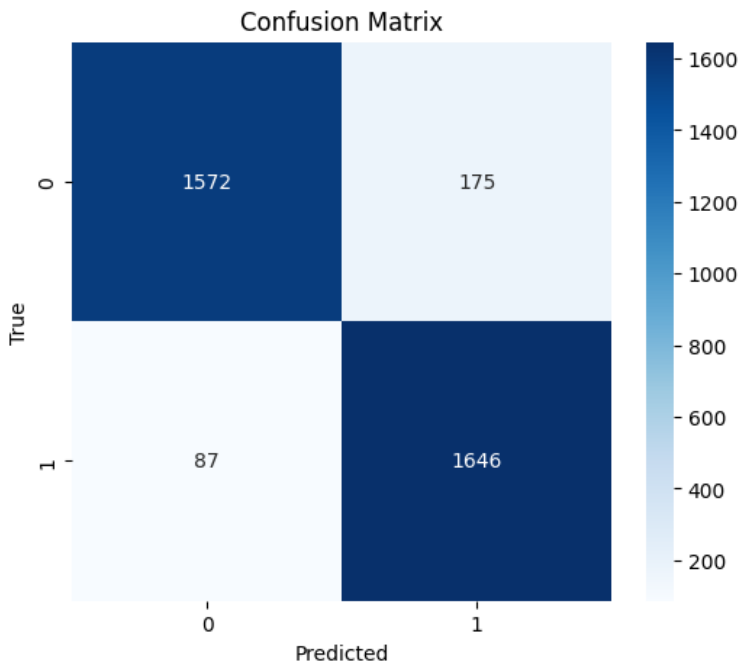

Figure 19: Confusion Matrix- ETT

To further demonstrate the performance of the model on the ETT dataset, predictions were generated for a subset of test samples (Figure 20). The predicted class labels closely matched the ground truth across the selected examples, indicating that the model successfully captured the underlying temporal patterns in the data. The consistency between true and predicted labels highlights the model's ability to generalize effectively beyond the training set and confirms its robustness when applied to real-world time series data.

The classification report for the ETT dataset is presented in Table 8. The model achieved an overall accuracy of 91 percent, with precision, recall, and F1-scores all averaging around 0.91 across both classes. Specifically, the classifier reached a precision of 0.87 and recall of 0.96 for class 0, while class 1 achieved a precision of 0.95 and recall of 0.86. This indicates that the model was highly effective at distinguishing between the two classes, though it tended to slightly favor identifying class 0 more reliably than class 1.

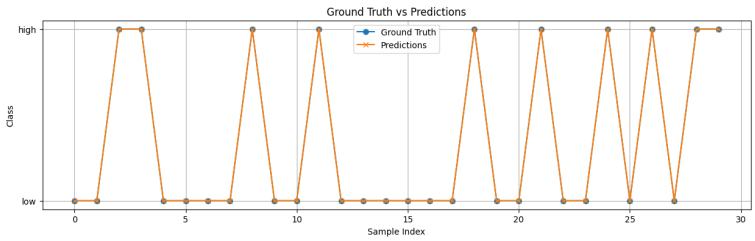

Figure 20: Sample predictions for 30 test inputs for ETT dataset

Compared to the synthetic dataset, where performance was perfect across all metrics, the results on the ETT dataset reflect the greater complexity and variability inherent in real-world time-series data. Nevertheless, the consistently high scores demonstrate that the model is capable of robust classification under realistic conditions, achieving a balanced trade-off between precision and recall for both classes.

Class	Precision	Recall	F1-score
0	.87	.96	.91
1	.95	.86	.90
Accuracy			.91

Table 8: Classification report of the experiment on ETT data.

6.2.2 *Learned Prototypes*

Figure 21 presents the ten prototypes discovered by the model from the ETT dataset. Each prototype summarizes a representative temporal pattern across the six features (HUFL, HULL, MUFL, MULL, LUFL, LULL) over a 24-hour horizon.

Several prototypes (e.g. P0, P1, P2) exhibit gradual rising and falling trends over the 24-hour horizon, reflecting smooth daily variations in load features. Others (e.g. P5, P6, P7) capture sharp fluctuations and abrupt transitions, indicating conditions of rapid change in the system. Prototypes such as P3 and P4 highlight divergent interactions between features, where one group of features increases while others decrease, showcasing the ability of the model to disentangle contrasting behaviors.

This variety demonstrates that the auxiliary clustering, evidence, and diversity losses successfully encouraged the model to learn non-redundant, interpretable prototypes. Instead of memorizing instances, the model constructed a compact set of representative patterns. These prototypes act as the building blocks for both classification and forecasting, allowing predictions to be explained in terms of meaningful temporal behaviors observed in the ETT data.

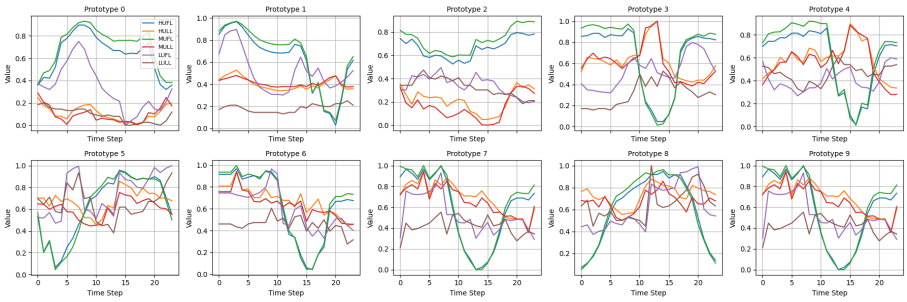

Figure 21: Representation of learned Prototypes

6.2.3 *Prototype Evaluation*

To evaluate the quality of the learned prototypes on the ETT dataset, hierarchical clustering was performed using DTW as the similarity measure. The evaluation was done by analyzing both intra-cluster and inter-cluster distances, along with the distribution of prototypes across clusters.

All clusters exhibited intra-cluster distances of 0.00, indicating that the prototypes grouped within each cluster were identical under the DTW metric. This outcome confirms that the prototypes within the same cluster capture highly consistent temporal patterns. For clusters consisting of a single prototype, the zero intra-cluster distance is trivial, while for multi-prototype clusters it reinforces the strong coherence of the grouped representations.

The inter-cluster distances reveal clear separation between clusters, the results (Table 9) shows values ranging from 19.30 (between Cluster 6 and Cluster 7) to 106.18 (between Cluster 1 and Cluster 7). The relatively high distances across most cluster pairs demonstrate that the prototypes are well separated, ensuring that different clusters encode distinct temporal dynamics. At the same time, smaller distances observed between some clusters (e.g., Cluster 6 and Cluster 7 with 19.30) suggest that certain prototypes share partially overlapping temporal characteristics, while still maintaining enough diversity to remain distinguishable.

Overall, the clustering analysis validates that the learned prototypes for the ETT dataset are compact within clusters and diverse across clusters, supporting their interpretability as meaningful representatives of different temporal behaviors in the data.

To further analyze the learned prototypes, we performed hierarchical clustering based on DTW distances between prototypes. The DTW matrix and corelation matrix helps to analyse and evaluate the learned prototypes.

The DTW distance matrix provides insights into the temporal similarity between learned prototypes. Each entry in the matrix quantifies how closely two prototypes align in terms of their temporal dynamics, even when shifts or distortions in time occur. As shown

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
C1	–	31.33	62.36	81.18	83.60	98.60	106.18	98.88	73.73	81.52
C2		–	61.31	77.51	78.32	95.79	102.89	97.93	74.73	85.68
C3			–	62.36	52.56	66.58	76.57	69.54	73.77	62.50
C4				–	23.97	39.00	39.39	40.28	55.88	70.51
C5					–	40.37	39.39	38.02	51.13	59.14
C6						–	19.30	23.81	52.99	74.44
C7							–	30.60	56.58	67.89
C8								–	49.21	70.54
C9									–	61.67
C10										–

Table 9: Upper-triangular inter-cluster DTW distances between medoids of prototype clusters on the ETT dataset. Entries below the diagonal are omitted due to symmetry.

in (figure 22), the DTW values reveal clear grouping patterns, with some prototypes exhibiting low distances (indicating highly similar temporal structures), while others remain well separated with higher distances. This separation is desirable, as it confirms that the learned prototypes capture diverse and non-redundant temporal behaviors present in the ETT dataset. The diagonal entries naturally remain zero, while off-diagonal values highlight relative dissimilarities.

In addition to temporal alignment, a correlation analysis was conducted among the prototypes to investigate their linear dependencies. The correlation matrix (figure 23) reveals how strongly each prototype relates to the others in terms of overall temporal patterns. High correlation values between certain pairs suggest that those prototypes capture overlapping trends, whereas weaker correlations indicate that they encode complementary or independent dynamics. Together with the DTW analysis, the correlation matrix provides a complementary view of prototype diversity, confirming that the set of learned prototypes balances both similarity and independence, thereby enhancing interpretability and robustness of the model.

The resulting dendrogram (Figure 24) reveals the hierarchical structure among prototypes, where closely related prototypes are grouped at smaller distances, while more distinct prototypes are merged higher in the hierarchy.

By applying a distance threshold, we obtained a cluster assignment for the prototypes. Within each cluster, we identified medoids, the most representative prototypes using a minimum total intra-cluster distance criterion. These medoids serve as concise representatives of the learned temporal patterns, providing a more interpretable summary of prototype space.

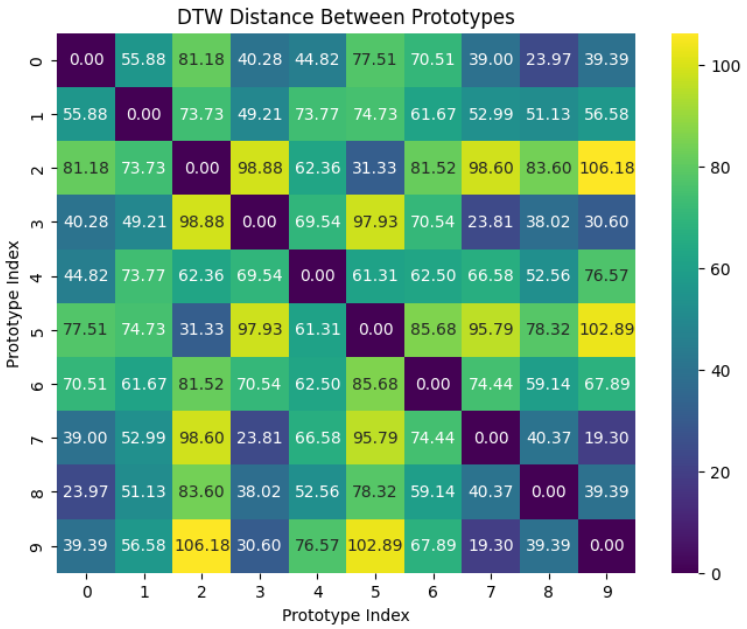

Figure 22: DTW Matrix of performing classification on ETT dataset

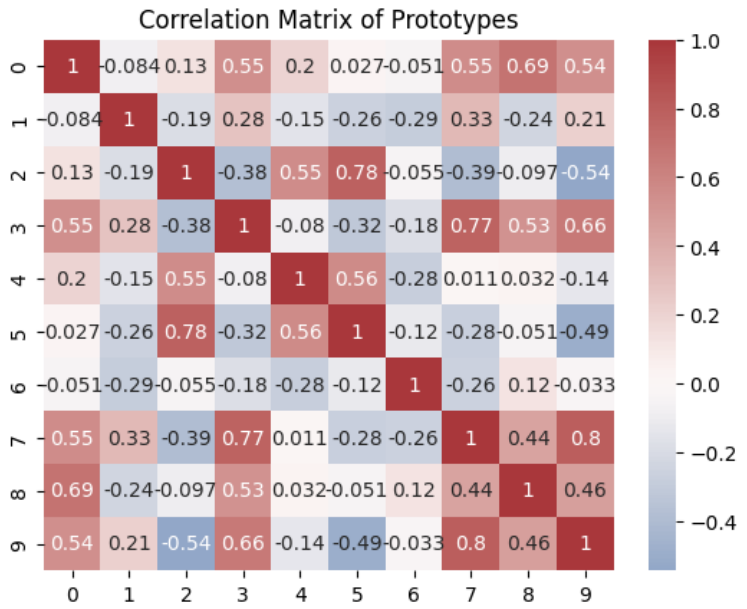

Figure 23: Correlation Matrix of prototypes on ETT dataset

This clustering analysis demonstrates that the model does not simply learn arbitrary prototypes, but instead captures structured groups of temporal dynamics. Such insights are valuable for both interpretability and downstream tasks, as they highlight the diversity and relationships among the prototypes learned during training.

After this analysis, redundant prototypes were removed and the model was updated with distinct prototypes for final evaluation on the test dataset. So, the redundant prototype is 9 and which is re-

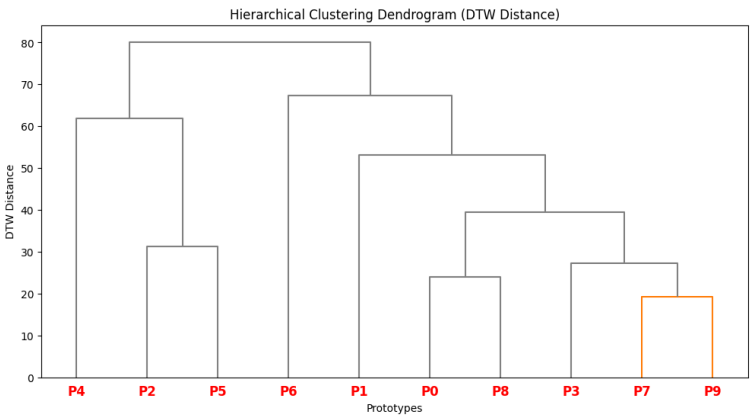

Figure 24: Dendrogram of performing classification on ETT dataset

moved and the model gets updated with rest of 9 distinct prototypes [0, 1, 2, 3, 4, 5, 6, 7, 8]. Fig. 21 gives a visual representation of the learned prototypes. From this, we can identify that prototype 7 and prototype 9 are similar.

6.2.4 *Sample Prediction and Prototype Similarity*

To further demonstrate the interpretability of the model, we analyze an individual test sample alongside the most similar learned prototype. Figure 25 shows the input sequence from the ETT dataset and its closest prototype identified by the model, which is Prototype 5. The temporal dynamics shows strong similarities across multiple features, particularly in the fluctuations observed around the middle of the sequence. For example, both the sample and Prototype 5 display consistent rises and falls in HUFL and MUFL, while other features remain relatively stable. This close resemblance highlights how the model grounds its prediction in representative prototypes, enabling transparent reasoning about why a particular prediction was made. Even when there are minor deviations in the input sequence, the prototype still serves as a meaningful reference, capturing the characteristic behavior patterns that drive the model decision.

6.2.5 *LLM Generated Explanation of Predictions*

To further enhance interpretability, we used the prototype-based reasoning mechanism to generate a textual explanation for the prediction. The explanation produced by the model for the given sample is reproduced below:

> *The model predicted class "High" for the given instance because it identified a strong similarity with the 5th nearest prototype. This similarity was attributed to the power load patterns in the*

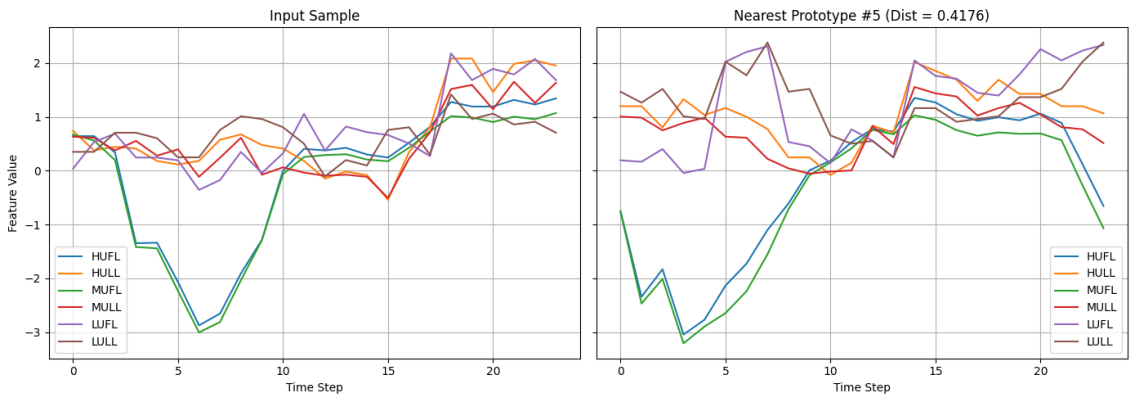

Figure 25: Representation of input sample and nearest prototype

input sequence that resembled the 'LUFL' and 'LULL' values of the prototype. These features indicate a high load on the transformer, which is an essential factor in determining the class. The temporal dynamics of input at specific times of the day is consistent with the 'HUFL' value of the prototype, further solidifying the prediction.

6.2.6 *Evaluation of Text Explanation*

The generated explanation consists of 86 words, which provides a reasonably detailed narrative of the model's reasoning. The completeness score is 1, suggesting that the explanation successfully highlighted key features. The explanation contains 2 minor grammatical errors, indicating mostly fluent text with only slight issues in readability. The readability score of 41.12 places the explanation at a moderately accessible level. Overall, the explanation demonstrates clarity and reasonable alignment with the model's decision.

6.3 FORECASTING RESULTS

In this section, we present the results of the forecasting task performed on the ETT dataset using the ProSeNet forecasting model. The model aims to predict the Oil Temperature (target) for the next 24 time steps, and generate interpretability insights through prototype-based explanations.

6.3.1 *Model Performance Evaluation*

Using the best-performing configuration identified through hyperparameter tuning, the ProSeNet model was retrained and evaluated on a test set. The optimal hyperparameter settings were in 10:

Table 10: Optimal hyperparameter configuration used for final evaluation.

Prototypes (k)	Learning Rate	λ_c	λ_e	λ_d
5	0.001	0.1	0.1	0.05

6.3.1.1 *Training and Validation Loss*

Figure 26 shows the loss curves for both training and validation sets over 100 epochs, providing insights into the convergence behavior of the ProSeNet forecasting model. The training loss decreases steadily throughout the epochs, indicating that the model is able to fit the data effectively. The validation loss follows a similar downward trend, though with higher variance, which is expected due to the stochastic nature of forecasting on unseen samples.

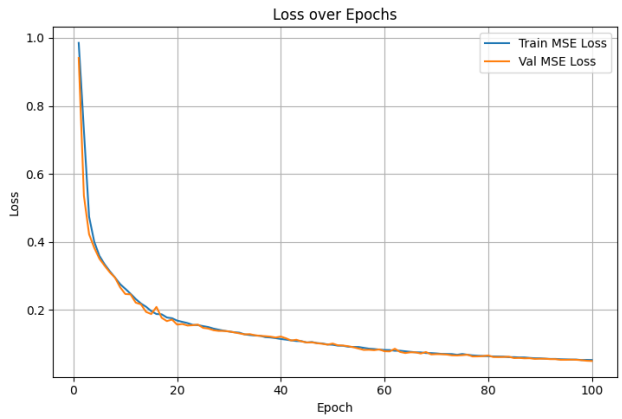

Figure 26: Training and validation loss curves for the ProSeNet forecasting model.

6.3.1.2 *Auxiliary Loss Analysis*

Figure 27 shows the auxiliary losses (clustering, evidence, and diversity loss) together with the MSE loss and total loss during the training of the ProSeNet forecasting model. These components play a crucial role in guiding prototype learning toward meaningful and interpretable representations.

The clustering loss decreases rapidly within the first 20 epochs and approaches zero, showing that instances are effectively matched to their closest prototypes early in training. Similarly, the evidence loss follows a sharp downward trend, reflecting the model's rapid ability to extract reliable and discriminative temporal patterns from the data. The diversity loss, while reducing more gradually, maintains a non-zero value throughout training, encouraging the prototypes to remain distinct and preventing redundancy.

The total loss curve mirrors the combined effect of these auxiliary objectives with the MSE loss, steadily converging as training

progresses. Importantly, the simultaneous reduction across all auxiliary components confirms that the regularization strategies successfully complement the forecasting objective, leading to stable prototype learning and enhancing interpretability without compromising predictive accuracy.

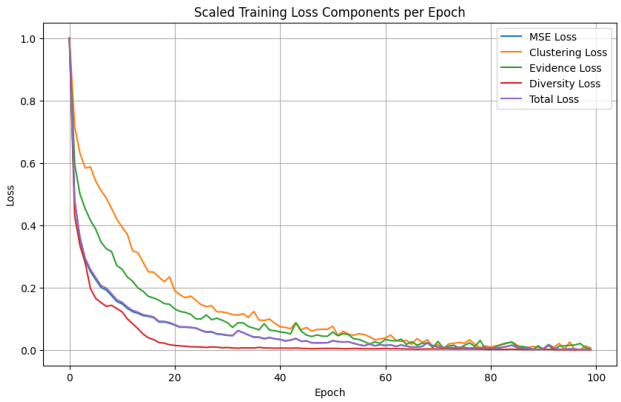

Figure 27: Auxiliary losses of forecasting(clustering, evidence, diversity) during training.

6.3.1.3 *Quantitative Evaluation*

Table 11 presents the forecasting performance in terms of common regression metrics including MSE, RMSE, MAE, and R^2. These metrics provide an overall measure of model accuracy and robustness.

Table 11: Forecasting performance metrics of ProSeNet model.

Metric	MSE	RMSE	MAE	R^2
	1.4232	1.1930	0.8371	0.9804

The model achieved a coefficient of determination of $R^2 = 0.9804$, indicating that it explains over 98% of the variance in the target variable. The low MSE and MAE values confirm the model's strong forecasting accuracy and stability.

6.3.2 *Prototype Evaluation*

6.3.2.1 *Visual Representation of Learned Prototypes*

Figure 28 presents the time-series representations of the 5 learned prototypes, which capture distinct temporal dynamics relevant to oil temperature forecasting.

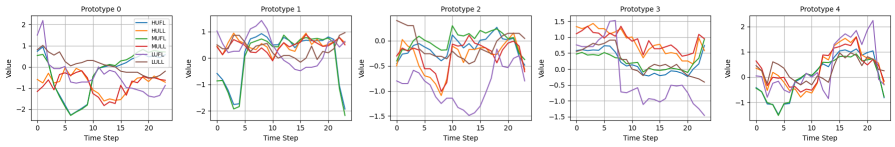

Figure 28: Visualization of the learned prototypes.

6.3.2.2 *DTW Distance Matrix*

Figure 29 presents the pairwise DTW distances computed between all prototype sequences identified by the model. Each element s_{ij} in the symmetric matrix $\mathbf{S}$ denotes the DTW distance between prototype P_i and prototype P_j. A smaller value of s_{ij} indicates a higher similarity in temporal patterns between the two prototypes, while larger values correspond to greater dissimilarity.

In this context, the DTW matrix serves as an important diagnostic tool for prototype diversity and redundancy analysis. The diagonal elements ($s_{ii} = 0$) represent self-similarity, while the off-diagonal elements quantify the temporal alignment cost between different prototypes. By examining this matrix, we can assess whether the prototypes learned by the model capture distinct temporal behaviors or if some prototypes are redundant or overlapping.

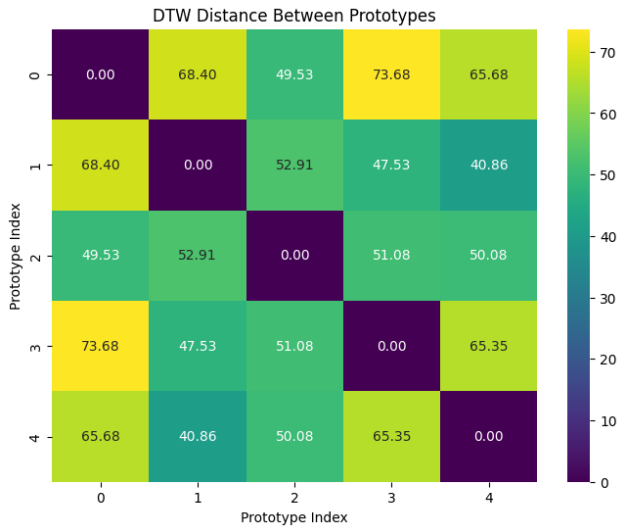

Figure 29: DTW matrix of learned prototypes.

For instance, in the DTW matrix, prototype pairs such as (P_1, P_4) and (P_1, P_3) exhibit relatively low DTW distances (approximately 40-47), suggesting that these prototypes encode similar temporal dynamics. In contrast, higher distances such as $(P_0, P_3) \approx 73.7$ indicate more dissimilar time-series patterns, implying that these prototypes represent diverse regions of the feature space.

Overall, the DTW distance matrix provides quantitative evidence of the representativeness and separability of the selected prototypes. A well-structured prototype set should balance low intra-prototype redundancy (high off-diagonal distances) with adequate coverage of the temporal variations present in the dataset.

6.3.2.3 *Correlation Matrix*

Figure 30 shows the correlation matrix $\mathbf{C}$ computed between the learned prototype representations. Each element c_{ij} indicates the Pearson correlation coefficient between prototype P_i and prototype P_j. Since $\mathbf{C}$ is symmetric with unit diagonal ($c_{ii} = 1$), the off-diagonal elements capture the linear dependency between different prototype embeddings.

From a prototype evaluation perspective, the correlation matrix provides insight into the *redundancy* and *diversity* among the learned prototypes. Low or near-zero correlations between prototypes (e.g., $c_{01} = -0.11$, $c_{03} = -0.04$) suggest that they capture distinct and uncorrelated temporal patterns in the dataset, indicating high representational diversity. Conversely, moderately positive correlations such as $c_{14} = 0.39$ or $c_{24} = 0.20$ imply that certain prototypes may share similar latent dynamics or partially overlapping temporal features.

In general, a well-structured prototype set should exhibit low inter-prototype correlations, ensuring that each prototype contributes unique information to the overall model. This matrix thus serves as a complementary measure to the DTW distance matrix, together providing a comprehensive evaluation of prototype distinctiveness, redundancy, and representational balance.

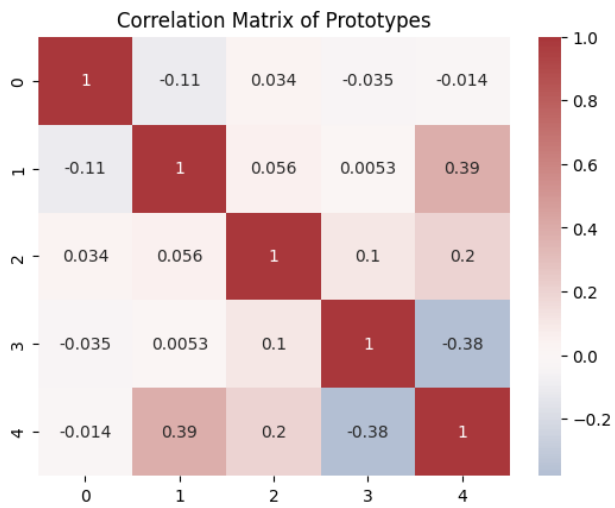

Figure 30: Correlation matrix between forecasting prototypes, highlighting linear relationships and contrasting behaviors.

6.3.2.4 *Cluster-based Distance Analysis*

Table 12: Inter-cluster DTW distances between cluster medoids of learned prototypes

Clusters	C_1	C_2	C_3	C_4	C_5
C_1	0.00	49.53	68.40	65.68	73.68
C_2		0.00	52.91	50.08	51.08
C_3			0.00	40.86	47.53
C_4				0.00	65.35
C_5					0.00

To further analyze the relationships between the learned prototypes, hierarchical clustering was performed on the DTW distance matrix derived from the five learned prototypes to analyze their structural relationships. Each prototype was assigned to a distinct cluster, resulting in five clusters (C_1 through C_5), with corresponding medoids $\{P_0, P_1, P_2, P_3, P_4\}$.

The *intra-cluster distances* for all clusters were found to be zero, as each cluster contained a single prototype (the medoid itself). This indicates that the learned prototypes are highly distinct and non-redundant with respect to one another.

The *inter-cluster DTW distances* shown in Table 12 quantify the temporal dissimilarity between cluster medoids. Larger DTW distances (e.g., between C_1 and C_5 with 73.68, or C_1 and C_3 with 68.40) indicate that the corresponding prototypes capture substantially different temporal dynamics. In contrast, moderate distances such as C_3–C_4 (40.86) and C_3–C_5 (47.53) suggest partial overlap in the temporal characteristics of those prototypes.

To further illustrate the relationships between the learned prototype clusters, we employ *t-distributed Stochastic Neighbor Embedding* (t-SNE) to project the DTW-based inter-cluster distance structure into a two-dimensional space, as shown in Figure 31. t-SNE is a nonlinear dimensionality reduction technique designed to preserve local neighborhood relationships when mapping high-dimensional dissimilarities into a low-dimensional embedding.

In this visualization, each point corresponds to a prototype cluster medoid, and the spatial proximity between points reflects their relative temporal similarity. Clusters positioned closer together indicate more similar temporal dynamics, while clusters that are farther apart represent more distinct prototype behaviors. For example, the pronounced separation between clusters C1 and C5, as well as C1 and

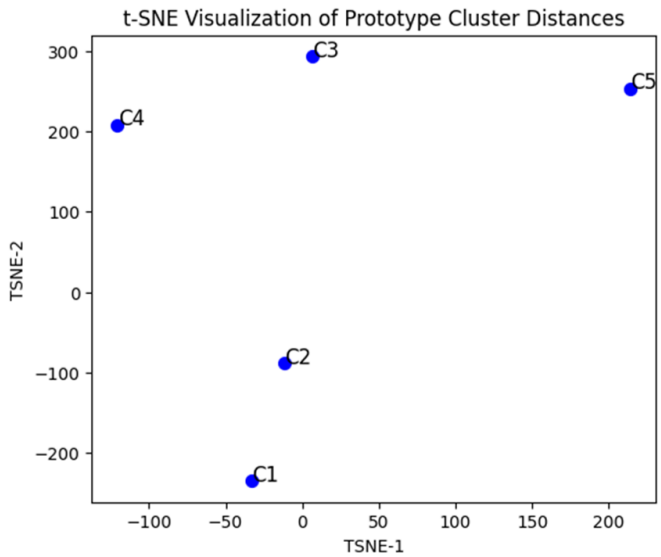

Figure 31: t-SNE visualization of inter-cluster relationships between prototype medoids based on DTW distances. Each point represents a prototype cluster medoid (C1–C5), and the relative proximity reflects similarity in temporal dynamics.

C3, visually corroborates their large DTW distances reported in Table 12. In contrast, clusters such as C3 and C4 appear closer in the embedding space, consistent with their smaller DTW distance (40.86), suggesting partial overlap in their temporal characteristics.

It is important to note that t-SNE emphasizes local structure rather than preserving exact global distances; therefore, the embedding should be interpreted qualitatively rather than quantitatively. Nevertheless, the clear spatial separation observed between most prototype clusters supports the conclusions drawn from the DTW-based inter-cluster analysis. Together, these results indicate that the model learns a diverse and well-separated set of prototypes, enhancing both the interpretability and representational coverage of the proposed prototype-based forecasting framework.

From a prototype evaluation perspective, the absence of intra-cluster variation and the relatively high inter-cluster distances together demonstrate that the model successfully learned a diverse set of prototypes. These prototypes represent well-separated temporal patterns, enhancing the model's ability to cover a wide range of behaviors within the time series and improving the overall interpretability of the learned representations.

6.3.3 *Prototype Utilization Analysis*

Figure 32 summarizes the utilization frequency of each learned prototype, indicating how many training samples were most strongly asso-

ciated with each prototype during model inference. Prototype utilization provides a quantitative measure of how effectively the learned prototypes are used in representing the input space.

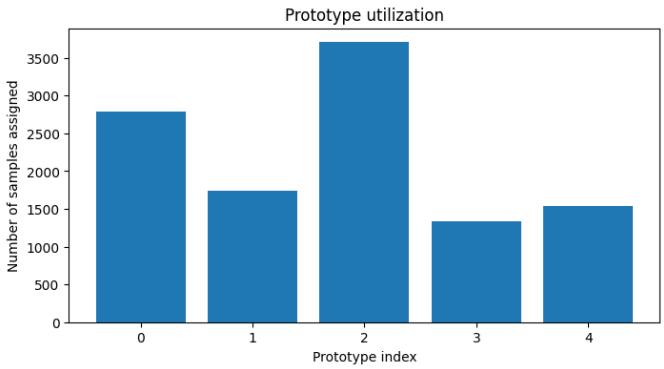

Figure 32: Prototype utilization summary showing the number of samples assigned to each prototype.

During model training, an *evidence loss* term was incorporated to regularize the prototype learning process. This loss component ensures that every prototype remains useful by penalizing configurations where one or more prototypes fail to represent any training samples. Without this constraint, the optimization process could lead to prototype collapse, where certain prototypes become inactive or unused. The evidence loss therefore encourages a balanced utilization across prototypes, promoting diversity and interpretability by forcing each prototype to correspond to at least one region of the input embedding space.

The prototype utilization was computed by passing the dataset through the trained model and recording, for each input sequence, which prototype achieved the highest similarity score. Specifically, for every sample x_i, the model computes a similarity vector $\mathbf{s}_i = [s_{i1}, s_{i2}, \ldots, s_{iK}]$ between the input and all K prototypes. The prototype with the maximum similarity,

$$p_i = \arg\max_k s_{ik},$$

is considered the assigned prototype for that sample. The final utilization count for each prototype P_k is obtained by summing all assignments where $p_i = k$. This procedure yields the distribution of prototype usage across the dataset and validates the effectiveness of the evidence loss in maintaining active participation of all prototypes.

6.3.4 *Sample Prediction and Prototype Similarity*

Finally, the interpretability of the forecasting task is demonstrated by generating textual explanations for predictions using the nearest prototypes among the learned representations.

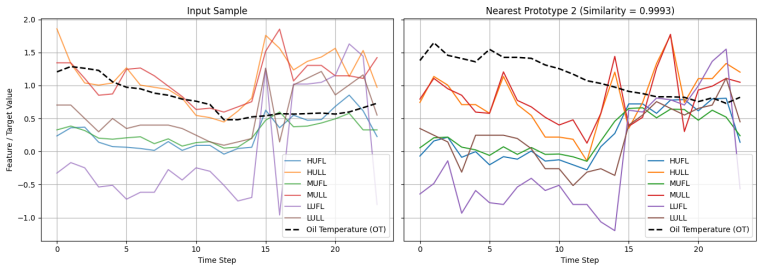

Figure 33: Comparison between a sample time-series sequence and its most similar prototype. The similarity score between them is 0.9993.

Figure 33 illustrates the alignment between a representative sample sequence and its nearest prototype identified through the prototype-based similarity scores. The purpose of this visualization is to demonstrate how the model captures and retrieves historically similar temporal patterns during inference.

As shown in Figure 33, both the sample and its nearest prototype exhibit closely matching temporal trajectories across all features. Particularly, the oil temperature feature (target) follows a rising trend between time steps 15-23, corresponding to an increase in transformer load and thermal buildup. The prototype sequence mirrors this progression with almost identical fluctuations, reflecting the model's ability to recognize similar operational patterns over time.

The exceptionally high similarity score (0.9993) indicates that the sample sequence aligns almost perfectly with the temporal dynamics encoded by its nearest prototype. This alignment suggests that the model effectively retrieves a historical representative pattern that closely matches the current operational state, validating the interpretability and reliability of the learned prototypes. In essence, the plot provides an interpretable, case-based explanation of the forecast by showing that the model's reasoning for this sample is grounded in a highly similar past sequence.
Figure 34 illustrates the temporal alignment between the oil temperature (OT) of a sample sequence (ground truth), the model's forecast, and the corresponding prototype sequence in the original temperature scale (°C). The oil temperature values range approximately from 16.9 °C to 27.5 °C, capturing realistic thermal variations within the transformer's operating cycle.

The *sample ground-truth sequence* begins with a relatively high temperature around 24.4 °C and gradually cools down to a minimum of approximately 17.5 °C near the middle of the time window. Toward the final time steps, the temperature rises again, stabilizing near 19.6 °C.

The *forecasted oil temperature* produced by the model closely follows the observed sample trend throughout the sequence. The forecast starts around 22.1 °C and shows a smooth decline to about 17.0 °C,

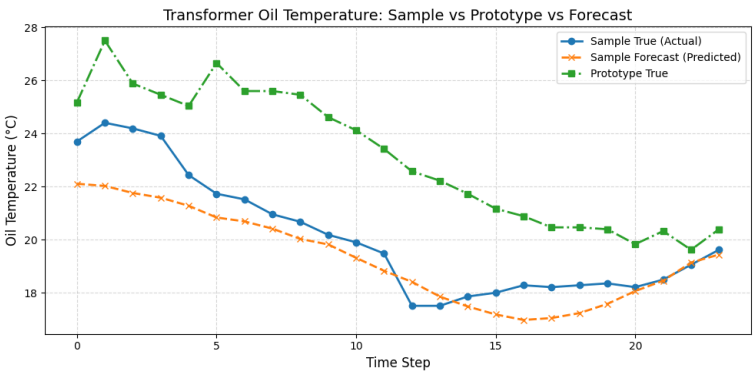

Figure 34: Comparison of predicted oil temperature (OT) for a sample sequence with its true OT values and the true OT pattern of the nearest prototype. The prototype similarity score is 0.9993.

before recovering gradually to approximately 19.4 °C. This trajectory demonstrates the model's ability to capture the cooling-warming dynamics of the transformer and reproduce the turning points of the ground-truth temperature profile with high temporal fidelity.

The *nearest prototype sequence* exhibits a highly similar temporal structure to both the sample and the forecast. It starts at a higher temperature near 27.5 °C, steadily cools down to around 19.8 °C, and then stabilizes near 20.4 °C toward the end. Despite the higher overall temperature range, the prototype mirrors the same cooling and recovery trend, demonstrating that the model successfully identifies a historically analogous thermal pattern from past data.

The near-perfect similarity score (0.9993) quantitatively confirms that the retrieved prototype encodes a temporal evolution nearly identical to that of the sample sequence. This strong alignment among the sample, forecast, and prototype validates that the model not only predicts accurate temperature trajectories but also grounds its reasoning in previously observed patterns, enhancing both performance and interpretability.

6.3.5 *LLM Generated Explanation of Predictions*

The explanation generated by the Large Language Model (LLM) provides a human-understandable summary of the model's reasoning behind its prediction.

6.3.5.1 *Generated Explanation (LLM Output)*

> *The oil temperature of the transformer is generally steady and smooth, showing a gentle decrease over time. The temperature*

follows a consistent pattern, with minor fluctuations that do not significantly impact the overall trend.

The closest prototype shows a similar trend, highlighting a similar temperature profile. The sample sequence and prototype share a high similarity, indicating that the temperature is stable and not experiencing any unusual changes or spikes.

The transformer is in a stable condition, with the oil temperature consistently following a predictable pattern.

6.3.5.2 *Interpretation and Practical Relevance*

The above explanation, generated by LLM, provides a clear and human-understandable summary of the model's reasoning. It describes the thermal condition of the transformer in plain language, emphasizing that the oil temperature is stable and follows a predictable pattern over time. In this case, the LLM highlights that both the sample and its closest prototype exhibit a smooth and consistent temperature profile, indicating that the transformer is operating under normal conditions without abrupt thermal variations or instability.

Such an explanation is particularly valuable for non-technical users, including transformer operators, maintenance engineers, and field technicians. Instead of interpreting complex numerical similarity scores or model outputs, they can easily understand from this description that the transformer's oil temperature is within a normal operating range. This helps maintenance personnel quickly infer that the system is not under thermal stress, there are no sudden temperature rises indicating potential faults, and the transformer is functioning as expected.

By converting quantitative prototype-based reasoning into clear natural language, the LLM effectively bridges the gap between advanced model interpretability and practical field diagnostics. This approach enhances the transparency and usability of the forecasting model, ensuring that insights derived from complex neural computations can directly support condition monitoring, preventive maintenance, and informed operational decision-making.

6.3.6 *Evaluation of Text Explanation*

The quality of the LLM-generated text explanation was evaluated using several quantitative metrics that assess its structure, clarity, and linguistic quality. Table 13 summarizes the computed values.

The explanation length of 101 words suggests that the model produced a concise and focused summary, providing sufficient contextual information without unnecessary complexity. Although the com-

Table 13: Quantitative metrics for evaluating the quality of the generated text explanation.

Metric	Value	Interpretation Direction
Length (words)	101	Moderate length indicates concise yet detailed text.
Completeness Score (0-2)	0	Higher is better; 0 indicates essential but minimal detail.
Fluency / Grammar Errors (↓ better)	0	No grammatical or structural errors detected.
Readability Score (↑ better)	17.13	High readability; easy for non-technical users.

pleteness score of 0 indicates that the explanation does not include detailed technical reasoning, it still captures the core message regarding the oil temperature's stability and its similarity to the prototype, which is appropriate for non-technical audiences.

The text achieved a fluency error count of 0, showing that it is free from grammatical or syntactic issues, and therefore maintains a natural and professional tone. Furthermore, the readability score of 17.13 reflects that the explanation is easily understandable, making it well-suited for transformer operators, maintenance engineers, and field technicians who may not have expertise in machine learning.

Overall, these results confirm that the LLM-generated explanation is clear, fluent, and user-friendly. While not exhaustive in technical depth, it effectively fulfills its interpretability objective by presenting the model's reasoning in a transparent and accessible manner that supports real-world decision-making and system monitoring.

6.4 MULTI-MODAL MODEL EVALUATION RESULTS

Following the baseline evaluation of the unimodal ProSeNet Regressor, a multi-modal variant was evaluated by incorporating textual embeddings alongside time-series inputs. The purpose of this experiment was to assess whether the inclusion of semantic information extracted from domain text data could enhance forecasting accuracy.

The model was trained using the same data split and optimal hyperparameter configuration obtained from the previous tuning experiments. Evaluation was performed on the test dataset across 10 independent runs to assess the stability and robustness of the predictions. The performance metrics for each run, along with the mean and standard deviation, are summarized in Tables 14and 15. The results in Table 14 show the test performance of the model over 10 independent runs. Across all runs, the coefficient of determination remains

consistently high, ranging from 0.9837 to 0.9883, indicating that the model is able to capture most of the variance in the test data. While the MSE and RMSE values exhibit some variability, particularly in Run 2 and Run 9, the overall performance is stable. Table 15 summarizes the mean and standard deviation for each metric across the 10 runs and demonstrating that the model not only provides accurate forecasts but also maintains reliable and robust performance across multiple training iterations.

It can be observed from the results that the incorporation of textual embeddings leads to consistent improvements across all performance metrics. The textual information allows the model to capture complementary patterns not present in the numerical sequences alone. Consequently, the multi-modal fusion enhances both predictive precision and generalization on unseen data. Furthermore, the low standard deviation of the metrics across multiple runs confirms that the model achieves stable and reliable performance. These findings indicate that the addition of text embeddings contributes to a better-structured latent representation space, supporting more interpretable and accurate forecasts.

Table 14: Test Metrics for All 10 Runs of ProSeNetRegressorWithText

Run	MSE	RMSE	MAE	R^2
1	0.8612	0.9280	0.6485	0.9881
2	1.4894	1.2204	0.8753	0.9795
3	0.8662	0.9307	0.6710	0.9881
4	0.8609	0.9279	0.6606	0.9881
5	0.8997	0.9485	0.6843	0.9876
6	0.9337	0.9663	0.6948	0.9871
7	0.9946	0.9973	0.6974	0.9863
8	0.8517	0.9229	0.6668	0.9883
9	1.1800	1.0863	0.7889	0.9837
10	0.9499	0.9746	0.6906	0.9869

Table 15: Summary of Test Metrics Across 10 Runs (Mean $\pm$ Std)

Metric	Mean	Std
MSE	0.9887	0.1913
RMSE	0.9903	0.0898
MAE	0.7078	0.0667
R^2	0.9864	0.0026

6.5 MODEL COMPARISONS

To assess the effectiveness of the proposed approach, the performance of the ProSeNet + Text model was compared against several baseline architectures, including standard recurrent, convolutional, and attention-based networks. All the baseline models were implemented with the same input segmentation, data normalization, and training setup as the proposed model to ensure a fair comparison. After training, models were evaluated on the same held-out test set using standard regression metrics: MSE, RMSE, MAE, and R^2.

Table 16 presents the performance comparison between the baseline and the proposed model.

Table 16: Performance comparison of different models on the test set.

Model	MSE	RMSE	MAE	R^2
LSTM	3.7952	1.9481	1.3228	0.9477
GRU	2.0289	1.4244	1.0532	0.9720
CNN–LSTM	2.0458	1.4303	1.0674	0.9722
Transformer	2.8685	1.6937	1.2662	0.9604
ProSeNet (No Text)	1.4232	1.1930	0.8371	0.9804
ProSeNet + Text (Proposed)	0.9887	0.9903	0.7078	0.9864

The proposed model outperformed the baseline across all evaluation metrics, achieving a lower prediction error and higher R^2 score. This demonstrates the advantage of integrating prototype-based reasoning and text-driven semantic alignment into time series modeling. The results demonstrate that incorporating prototype reasoning and text embeddings significantly enhances predictive performance and interpretability.

7 DISCUSSION AND CONCLUSION

7.1 DISCUSSION

This section reflects on the research questions posed in Chapter 1 and discusses the extent to which the conducted experiments provide answers.

RQ1: How can prototype-based explanations be utilized to enhance the interpretability of time-series forecasting models?

The experimental results demonstrate that prototype-based explanations provide an effective mechanism for enhancing interpretability in time-series forecasting. By learning representative temporal prototypes in a latent embedding space, the model enables case-based reasoning where forecasts are explained through similarity to historical patterns rather than abstract internal representations. The clustering, medoid selection, and pruning strategy further improves interpretability by reducing redundancy among prototypes and ensuring that each prototype captures a distinct temporal behavior. Across both synthetic and real-world datasets, prototype utilization analysis showed that predictions consistently align with meaningful and recurring patterns, supporting transparency and user trust in the forecasting process.

RQ2: How can prompt engineering be applied to enable LLMs to transform prototype explanations into human-readable texts?

The results indicate that prompt engineering plays a critical role in enabling large language models to effectively translate prototype-based explanations into natural language. By providing structured prompts containing prototype similarity scores, feature relevance, and contextual information, the LLM was able to generate coherent, readable, and informative textual explanations. Quantitative evaluation using length, completeness, fluency, and readability metrics confirmed that the generated explanations captured key explanatory features while remaining accessible to non-expert users. These findings suggest that LLMs can serve as a practical interface between technical prototype representations and human-understandable explanations, significantly improving the accessibility of explainable forecasting models.

RQ3: How might explanatory insights be leveraged to improve the model's predictive accuracy?

The experimental results suggest that prototype-based reasoning can indirectly support predictive performance. The alignment of latent embeddings with representative prototypes encourages the model to focus on robust and generalizable temporal patterns, acting as a form of implicit regularization. Forecasting experiments showed that the proposed framework achieves competitive performance compared to baseline deep learning models, indicating that the inclusion of explanation mechanisms does not degrade accuracy. Although direct performance gains from explanation generation alone are limited, the structured learning induced by prototype alignment contributes to stable training and consistent forecasting behavior.

7.2 CONCLUSION

This thesis investigated how prototype-based learning and multimodal reasoning can be applied to develop interpretable deep learning models for multivariate time-series forecasting. By extending prototype-based methods to the forecasting domain and integrating numerical time-series representations with natural language explanations, the proposed framework addresses the growing need for transparency and trust in predictive systems.

The results demonstrate that prototype-based reasoning provides an intuitive and effective explanation mechanism, grounding forecasts in representative historical patterns. The integration of a lightweight instruction-tuned language model enables these explanations to be communicated in human-readable form, bridging the gap between numerical inference and user understanding. Furthermore, extending the framework to a multimodal setting by incorporating textual embeddings enhances both predictive performance and contextual awareness.

Overall, this work shows that interpretability and forecasting accuracy need not be competing objectives. By reframing prototype-based explainability as a multimodal forecasting problem, the thesis contributes a practical and extensible approach for building transparent, trustworthy, and context-aware time-series prediction models.

7.3 LIMITATIONS AND FUTURE WORK

Despite the promising results, the proposed prototype-based framework has several limitations that should be acknowledged. First, the quality and interpretability of the explanations are strongly dependent on the quality, diversity, and placement of the learned proto-

types. If the number of prototypes is insufficient or if they are poorly distributed in the latent space, the resulting explanations may be less informative or overly redundant. Although clustering, medoid selection, and pruning were applied to mitigate this issue, prototype selection remains a sensitive component of the framework.

In addition, the proposed framework employs a fixed Gaussian kernel to transform Euclidean distances into similarity scores. While this choice provides a simple and interpretable similarity measure, it assumes a uniform distance scale across the latent space and may not optimally adapt to varying data distributions. This design reflects a trade-off between interpretability and modeling flexibility.

Second, the integration of a large language model introduces variability in explanation quality. While the generated explanations were generally coherent and informative, they may occasionally omit relevant details, include redundant information, or exhibit sensitivity to prompt design. Additionally, the evaluation of explanation quality relied on automated text metrics, which may not fully capture human perception of usefulness or trustworthiness.

Future work could address these limitations by exploring adaptive or learnable similarity formulations, such as kernel functions with trainable bandwidths or attention-based mechanisms to modulate prototype influence dynamically. Such extensions could improve flexibility while maintaining the interpretability benefits of prototype-based reasoning.

Beyond similarity modeling, future work could explore adaptive or attention-based prototype selection strategies, as well as jointly optimizing forecasting and explanation objectives during training. Incorporating human-in-the-loop evaluation, particularly with domain experts, would provide more reliable assessment of explanation quality and relevance. Extending the framework to larger and more diverse real-world datasets, and investigating alternative multimodal fusion strategies, could further improve generalization. Overall, these directions highlight the potential of the proposed framework as a foundation for continued research into interpretable and trustworthy time-series forecasting models, and suggest that prototype-based reasoning remains a promising paradigm for balancing predictive performance with transparency.

[1] Amina Adadi and Mohammed Berrada. Peeking inside the black-box: a survey on explainable artificial intelligence (xai). *IEEE access*, 6:52138–52160, 2018.

[2] Jatin Bedi and Durga Toshniwal. Deep learning framework to forecast electricity demand. *Applied energy*, 238:1312–1326, 2019.

[3] Ahsan Bilal, David Ebert, and Beiyu Lin. Llms for explainable ai: A comprehensive survey. *arXiv preprint arXiv:2504.00125*, 2025.

[4] Erik Cambria, Lorenzo Malandri, Fabio Mercorio, Navid Nobani, and Andrea Seveso. Xai meets llms: A survey of the relation between explainable ai and large language models. *arXiv preprint arXiv:2407.15248*, 2024.

[5] Alessandro Castelnovo, Roberto Depalmas, Fabio Mercorio, Nicolò Mombelli, Daniele Potertì, Antonio Serino, Andrea Seveso, Salvatore Sorrentino, and Laura Viola. Augmenting xai with llms: A case study in banking marketing recommendation. In *World Conference on Explainable Artificial Intelligence*, pages 211–229. Springer, 2024.

[6] Yue Cui, Jiandong Xie, and Kai Zheng. Historical inertia: A neglected but powerful baseline for long sequence time-series forecasting. In *Proceedings of the 30th ACM international conference on information & knowledge management*, pages 2965–2969, 2021.

[7] Subhajit Das, Panpan Xu, Zeng Dai, Alex Endert, and Liu Ren. Interpreting deep neural networks through prototype factorization. In *2020 International Conference on Data Mining Workshops (ICDMW)*, pages 448–457. IEEE, 2020.

[8] Jakob Droste, Ronja Fuchs, Hannah Deters, Jil Klünder, and Kurt Schneider. Explainability requirements for time series forecasts: a study in the energy domain. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, pages 229–239. IEEE, 2024.

[9] Renato Miranda Filho, Anísio M Lacerda, and Gisele L Pappa. Explainable regression via prototypes. *ACM Transactions on Evolutionary Learning*, 2(4):1–26, 2023.

[10] Alan H Gee, Diego Garcia-Olano, Joydeep Ghosh, and David Paydarfar. Explaining deep classification of time-series data with learned prototypes. In *CEUR workshop proceedings*, volume 2429, page 15, 2019.

[11] Alberto Guillén, Luis Javier Herrera, Ginés Rubio, Héctor Pomares, Amaury Lendasse, and Ignacio Rojas. New method for instance or prototype selection using mutual information in time series prediction. *Neurocomputing*, 73(10-12):2030–2038, 2010.

[12] Chao Huang, Zhao Kang, and Hong Wu. A prototype-based neural network for image anomaly detection and localization. *Neural Processing Letters*, 56(3):169, 2024.

[13] Qihan Huang, Mengqi Xue, Haofei Zhang, Jie Song, and Mingli Song. Is protopnet really explainable? evaluating and improving the interpretability of prototypes. *arXiv preprint arXiv:2212.05946*, 2022.

[14] Qihan Huang, Mengqi Xue, Wenqi Huang, Haofei Zhang, Jie Song, Yongcheng Jing, and Mingli Song. Evaluation and improvement of interpretability for self-explainable part-prototype networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2011–2020, 2023.

[15] Yushan Jiang, Wenchao Yu, Geon Lee, Dongjin Song, Kijung Shin, Wei Cheng, Yanchi Liu, and Haifeng Chen. Explainable multi-modal time series prediction with llm-in-the-loop. *arXiv preprint arXiv:2503.01013*, 2025.

[16] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728*, 2023.

[17] Arunas Kalinauskas and Dennis Slepov. Post-correction of time series forecasts. Master's thesis, Halmstad University, Halmstad, Sweden, 2023. URL https://urn.kb.se/resolve?urn=urn:nbn:se:hh:diva-51164. Retrieved from https://urn.kb.se/resolve?urn=urn:nbn:se:hh:diva-51164.

[18] Donald E. Knuth. Computer Programming as an Art. *Communications of the ACM*, 17(12):667–673, December 1974.

[19] Oscar Li, Hao Liu, Chaofan Chen, and Cynthia Rudin. Deep learning for case-based reasoning through prototypes: A neural network that explains its predictions. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[20] Wenxiang Li and K. L. Eddie Law. Deep learning models for time series forecasting: A review. *IEEE Access*, 12:92306–92327, 2024. doi: 10.1109/ACCESS.2024.3422528.

[21] Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: a survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.

[22] Xinhe Liu and Wenmin Wang. Deep time series forecasting models: A comprehensive survey. *Mathematics*, 12(10):1504, 2024.

[23] Xinhe Liu and Wenmin Wang. Deep time series forecasting models: A comprehensive survey. *Mathematics*, 12(10), 2024. ISSN 2227-7390. URL https://www.mdpi.com/2227-7390/12/10/1504.

[24] Scott Lundberg. A unified approach to interpreting model predictions. *arXiv preprint arXiv:1705.07874*, 2017.

[25] Ram Machlev, Leena Heistrene, Michael Perl, Kfir Yehuda Levy, Juri Belikov, Shie Mannor, and Yoash Levron. Explainable artificial intelligence (xai) techniques for energy and power systems: Review, challenges and opportunities. *Energy and AI*, 9:100169, 2022.

[26] Antonios Mamalakis, Elizabeth A Barnes, and Imme Ebert-Uphoff. Investigating the fidelity of explainable artificial intelligence methods for applications of convolutional neural networks in geoscience. *Artificial Intelligence for the Earth Systems*, 1(4):e220012, 2022.

[27] Philip Mavrepis, Georgios Makridis, Georgios Fatouros, Vasileios Koukos, Maria Margarita Separdani, and Dimosthenis Kyriazis. Xai for all: Can large language models simplify explainable ai? *arXiv preprint arXiv:2401.13110*, 2024.

[28] Orfeas Menis Mastromichalakis, Giorgos Filandrianos, Jason Liartis, Edmund Dervakos, and Giorgos Stamou. Semantic prototypes: Enhancing transparency without black boxes. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 1680–1688, 2024.

[29] Rada Mihalcea, Hugo Liu, and Henry Lieberman. Nlp (natural language processing) for nlp (natural language programming). In *Computational Linguistics and Intelligent Text Processing: 7th International Conference, CICLing 2006, Mexico City, Mexico, February 19-25, 2006. Proceedings 7*, pages 319–330. Springer, 2006.

[30] Yao Ming, Panpan Xu, Huamin Qu, and Liu Ren. Interpretable and steerable sequence learning via prototypes. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 903–913, 2019.

[31] Fuseini Mumuni and Alhassan Mumuni. Explainable artificial intelligence (xai): from inherent explainability to large language models. *arXiv preprint arXiv:2501.09967*, 2025.

[32] Anushka Narayanan and Karianne J Bergen. Prototype-based methods in explainable ai and emerging opportunities in the geosciences. *arXiv preprint arXiv:2410.19856*, 2024.

[33] Vikas C Raykar, Arindam Jati, Sumanta Mukherjee, Nupur Aggarwal, Kanthi Sarpatwar, Giridhar Ganapavarapu, and Roman Vaculin. Tsshap: Robust model agnostic feature-based explainability for time series forecasting. *arXiv preprint arXiv:2303.12316*, 2023.

[34] G Pradeep Reddy and YV Pavan Kumar. Explainable ai (xai): Explained. In *2023 IEEE Open Conference of Electrical, Electronic and Information Sciences (eStream)*, pages 1–6. IEEE, 2023.

[35] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. " why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1135–1144, 2016.

[36] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *https://doi.org/10.1038/s42256-019-0048-x*, 2019.

[37] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.

[38] A Saranya and R Subhashini. A systematic review of explainable artificial intelligence models and applications: Recent developments and future trends. *Decision analytics journal*, 7:100230, 2023.

[39] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: a systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608*, 2024.

[40] Chenxi Sun, Hongyan Li, Yaliang Li, and Shenda Hong. Test: Text prototype aligned embedding to activate llm's ability for time series. *arXiv preprint arXiv:2308.08241*, 2023.

[41] Mingtian Tan, Mike Merrill, Vinayak Gupta, Tim Althoff, and Tom Hartvigsen. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems*, 37:60162–60191, 2024.

[42] Andreas Theissler, Francesco Spinnato, Udo Schlegel, and Riccardo Guidotti. Explainable ai for time series classification: A review, taxonomy and research directions. *IEEE Access*, 10:100700–100724, 2022. doi: 10.1109/ACCESS.2022.3207765.

[43] Fabeena Thoufeeq and Reeba Susan Ashok. Mining infrequent group of motifs from multidimensional time series: A case study

at Alfa Laval AB. Master's thesis, Halmstad University, Halmstad, Sweden, 2024. URL https://urn.kb.se/resolve?urn=urn:nbn:se:hh:diva-52449. Retrieved from https://urn.kb.se/resolve?urn=urn:nbn:se:hh:diva-52449.

[44] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[45] WANG Xin, WU Ji, LIU Chao, YANG Haiyan, DU Yanli, and NIU Wensheng. Exploring lstm based recurrent neural network for failure time series prediction. *Journal of Beijing University of Aeronautics and Astronautics*, 44(4):772–784, 2018. doi: 10.13700/j.bh.1001-5965.2017.0285. URL https://bhxb.buaa.edu.cn/bhzk/en/article/doi/10.13700/j.bh.1001-5965.2017.0285.

[46] Janghoon Yang. Context information can be more important than reasoning for time series forecasting with a large language model. *arXiv preprint arXiv:2502.05699*, 2025.

[47] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI)*, page 11106â11115, 2021.

[48] Alexandra Zytek, Sara Pidò, and Kalyan Veeramachaneni. Llms for xai: Future directions for explaining explanations. *arXiv preprint arXiv:2405.06064*, 2024.

DECLARATION

Put your declaration here.

Halmstad, December 30, 2025

Anila Shaji, Meghna Vijay,
December 30, 2025

.

HALMSTAD UNIVERSITY

PO Box 823, SE-301 18 Halmstad
Phone: +35 46 16 71 00
E-mail: registrator@hh.se
www.hh.se